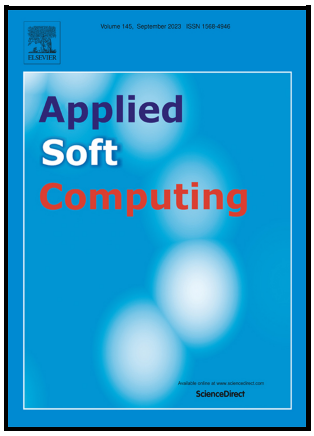
Journal Pre-proof

Artificial Bee Colony with Momentum-Guided Search and Starfish Exploration for Complex and Multimodal Problems

Harun Akbulut



PII: S1568-4946(26)01147-6

DOI: https://doi.org/10.1016/j.asoc.2026.115699

Reference: ASOC115699

To appear in: *Applied Soft Computing*

Received date: 17 March 2026
Revised date: 28 May 2026
Accepted date: 1 June 2026

Please cite this article as: Harun Akbulut, Artificial Bee Colony with Momentum-Guided Search and Starfish Exploration for Complex and Multimodal Problems, *Applied Soft Computing,* (2026) doi:https://doi.org/10.1016/j.asoc.2026.115699

This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2026 Published by Elsevier.

Journal Pre-proof

Artificial Bee Colony with Momentum-Guided Search and Starfish Exploration for Complex and Multimodal Problems

Harun Akbulut[*1]

[*1] Department of Artificial Intelligence and Machine Learning, Faculty of Computer and Information Sciences, Kayseri University, Türkiye

aharunakbulut@kayseri.edu.tr

ABSTRACT

The Artificial Bee Colony (ABC) algorithm is a widely used swarm intelligence method due to its simple structure and strong exploitation capability. However, it may experience performance loss in complex and multi-modal optimization problems due to its limited exploration capability and tendency toward early convergence. In this study, an optimization algorithm called Momentum-Guided Search and Starfish Exploration-based Artificial Bee Colony (MSE-ABC) is proposed to overcome these limitations. The proposed method preserves the observer and scout bee phases employed in the classical ABC algorithm while enhancing its global search capability with trigonometric exploration operators inspired by starfish optimization algorithm (SFOA). Furthermore, a momentum-based steering mechanism and dynamic exponential step size control are integrated to accelerate convergence during the search process and adaptively maintain the exploration-exploitation balance. The performance of the MSE-ABC algorithm was comprehensively evaluated through CEC 2022 benchmark tests, a five different real-world engineering design optimization problems, and deep learning hyperparameter optimization experiments conducted on the CIFAR-10 datasets. Experimental results demonstrate that the proposed method achieves superior or competitive performance when compared with eight state-of-the-art metaheuristic algorithms. The statistical significance of the obtained results was verified using the Wilcoxon signed-rank test and the Friedman test. The findings reveal that the MSE-ABC algorithm offers an effective and reliable optimization approach for complex and multi-modal optimization problems.

Keywords: Artificial Bee Colony, Starfish-Inspired Optimization, Hybrid Metaheuristic Optimization, Complex and Multimodal Optimization, Hyperparameter Optimization

1. Introduction

Optimization algorithms are numerical methods developed to find optimal solutions to complex problems encountered in a wide range of disciplines such as engineering, economics, logistics, data science, and finance, or to approximate these solutions as closely as possible [1]. These algorithms aim to optimize an objective function defined under certain constraints and are often designed to perform effectively in complex, multimodal, and nonlinear search spaces [2]. Today, the increasing scale of problems, the growing number of decision variables, and the increasingly complex structure of search spaces significantly complicate the optimization process. In this context, classical deterministic and derivative-based optimization approaches, such as gradient descent and Newton-based methods, often fall short in many real-world problems and frequently get stuck in local optimum solutions due to their reliance on strong mathematical assumptions such as continuity, differentiability, and convexity [3]. The loss of effectiveness of such methods in the face of noisy, discontinuous, or multi-modal objective functions has increased the need for alternative search strategies. To overcome these limitations, heuristic and metaheuristic optimization algorithms inspired by biological, physical, and social processes in nature have attracted considerable interest and been intensively researched over the past thirty years [4]. Metaheuristic algorithms offer more flexible and applicable alternatives compared to deterministic methods due to their ability to obtain solutions close to the global

optimum within reasonable computation times in large, complex search spaces, without requiring rigid assumptions about the problem structure or derivative information [5]. For this reason, meta-heuristic algorithms are widely applied not only in engineering and optimization problems, but also in a wide variety of fields such as healthcare, defense, finance, energy, transportation, telecommunications, and social sciences, with the aim of producing effective and efficient solutions [6-18].

In the literature, meta-heuristic algorithms are generally examined under two main categories: single-solution-based algorithms, which rely on the iterative improvement of a single solution candidate, and population-based algorithms, which are based on the simultaneous evolution of a population consisting of multiple candidate solutions [19]. Single-solution-based methods such as Simulated Annealing and Tabu Search stand out for their strong local search capabilities; population-based algorithms, on the other hand, can produce more effective results in multi-modal problems thanks to their advantage of being able to explore different regions of the solution space simultaneously. Swarm intelligence methods, which occupy an important place among population-based metaheuristic algorithms, perform searches by modeling the collective and cooperative behavior of agents such as particles, bees, ants, or flocks, aiming to naturally maintain the balance between exploration and exploitation [20]. Particle Swarm Optimization (PSO) [21], Ant Colony Optimization (ACO) [22], and Artificial Bee Colony (ABC) algorithm [23], are among the most widely used swarm intelligence-based approaches in the literature.

Real-world problems in modern engineering and science are often complex, multimodal, nonlinear, and NP-hard, making them very difficult to solve using traditional mathematical methods. While current meta-heuristic algorithms offer nature-inspired mechanisms to overcome this complexity, they face serious limitations such as early convergence, falling into local optimum traps, and an imbalance between exploration and exploitation capabilities in the search space. Although there are currently more than 540 meta-heuristic algorithms, the fact that, according to the “No Free Lunch” theorem, no algorithm can perform optimally on all problem types necessitates the continuous development of new methods [24]. Although the Artificial Bee Colony (ABC) algorithm stands out due to its simple structure and low parameter requirements, its standard version has low exploitation capacity because it updates only one dimension when generating new solutions [25]. To address these weaknesses, researchers are modifying the mechanisms of existing algorithms, particularly by integrating the information of globally optimal or elite individuals into the solution search equations [26]. Among the improved mechanisms are the adaptation of coefficients that guide the search process and the addition of modification rates that determine how many dimensions of the solution will be changed [27]. Furthermore, approaches that utilize distance information between individuals, rather than focusing solely on fitness values, increase convergence speed while preserving population diversity [28]. To prevent algorithms from becoming stagnant, feeding the scout bee phase from historical experience or opposition-based learning, rather than making it completely random, is a critical improvement. Such mechanical changes make the ABC algorithm more resilient in complex optimization landscapes [29]. As a result, replacing one-dimensional updates with multidimensional and guided strategies is essential to meet modern optimization needs. Enhancing the exploitation capabilities of these population-based methods enables the production of high-precision engineering solutions. Researchers continue to develop new approaches that dynamically adjust algorithm parameters to overcome these limitations.

Modified and improved models of the ABC algorithm focus on strengthening the algorithm's weak local search capabilities and increasing convergence speed. For example, the ABC-MNG (Multi-Neighbor Guidance) model balances by selecting not only the best neighbor for each individual but also the farthest and closest neighbors based on Euclidean distance. In this model,

population diversity is effectively preserved by using the cosine similarity metric to capture the directional information of candidate solutions [30]. Another important development, the MGABC approach, guides the search equations by establishing an elite group consisting of the top 10% of the population [31]. The ASRGABC model offers a mechanism that adaptively selects the appropriate search strategy by monitoring success rates in current and previous iterations. In this model, the population is randomly divided into groups, and scout bees are made to follow local leaders to avoid local optima [32]. The IABC model, designed for grid-based path planning problems, incorporates a path straightening strategy that eliminates unnecessary corners in the path. IABC also enhances global performance by incorporating a local search mechanism that optimizes the best solution at the end of each iteration [33]. In the ABCG variant, inspired by the gravity model, the attraction force between individuals is calculated, and neighbor selection is performed accordingly. In the improved models, the scout bee phase scans opposite regions using adversarial learning instead of generating completely random solutions. In variants developed for test scenario generation, the fitness function evaluates values in a four-level categorical structure. This categorization distinguishes between borderline-valid, valid, borderline-invalid, and invalid values, thereby improving exploitation quality [34]. Some advanced versions adaptively change the population size throughout iterations to minimize computational cost. Exploitation-focused speed equations that use the information of the global best individual (gbest) are frequently integrated into the local search phase of ABC in such models. These mechanical improvements enable ABC to produce accurate results in complex physical and engineering problems [35]. Neighborhood topologies and hierarchical leadership systems balance the algorithm's exploration capability with exploitation. Dynamic perturbation rates adjust the scanning sensitivity in the solution space on a problem-by-problem basis. As a result, the improved models exhibit much faster convergence performance than standard ABC.

A hybrid method is an integrated approach that aims to overcome the shortcomings of a single algorithm by combining the advantages of two or more meta-heuristic algorithms or different computational techniques. Among hybridization methods, it is common to blend one algorithm's global exploration capability with another's local exploitation power or to use machine learning for parameter tuning. The HABC-QL model combines the ABC algorithm with Q-learning to enable the adaptive selection of the optimal neighborhood search operator during the scout bee phase [36]. In a model used for gas pipeline optimization, the ABC algorithm is hybridized with Deep Reinforcement Learning (Actor-Critic), where a data-driven policy determines which dimensions to update. In this approach, the Actor network calculates update probabilities, while the Critic network evaluates the quality of these decisions to manage the algorithm [37]. The HABCSMO model combines ABC's exploration capability with the hierarchical leader-based learning mechanisms of the Spider Monkey Optimization (SMO) algorithm [29]. Similarly, the ABC-CSO (Chicken Swarm Optimization) hybrid maximizes energy efficiency in wireless networks by integrating chicken swarm behavior into ABC's cluster head selection process [38]. Another hybrid, ABC-SA, incorporates the Simulated Annealing temperature control mechanism into the optimization process, allowing initially poor solutions to be accepted. This enables the algorithm to escape local optima traps and explore a broader global search space [39]. Differential Evolution (DE) operators or Particle Swarm Optimization (PSO) velocity equations are also used in hybrid models to enhance exploitation capability [29]. Chaotic maps are incorporated into hybrid structures to make the initial population's randomness more balanced. Tabu Search (TS) mechanisms are used in these systems to prevent the repetition of previously searched regions. Reinforcement learning-based structures enable the algorithm to update its own strategy during operation. Function approximation methods improve solution accuracy in complex engineering problems involving continuous variables. As a result, hybrid structures create much more robust systems by

disciplining the random nature of metaheuristics with scientific rigor. These integrated approaches are considered among the most powerful tools in modern optimization.

Table 1. ABC algorithm and its variants

Algorithm	Year	Inspiration Source	Main Modification	Advantages	Limitations
Gbest-guided ABC (GABC) [40]	**2010**	ABC + global-best guidance	Global-best term added to search equation	Faster convergence; stronger exploitation	Reduced diversity; premature convergence risk
Modified ABC (MABC) [15]	**2010**	ABC with modified neighbor generation	New perturbation strategy	Improved local search; better robustness	Slightly higher complexity
PS-ABC (Hybrid ABC–PSO) [41]	**2015**	Bees + particle swarm dynamics	PSO velocity concept embedded into ABC	Faster convergence; improved exploitation	Parameter tuning required
hABCDE (Hybrid ABC with Differential Evolution) [42]	**2017**	ABC + DE operators	DE mutation integrated into ABC phases	Enhanced convergence accuracy; balanced search	Higher computational cost
ABCADE (Adaptive DE-based ABC) [43]	**2017**	ABC + adaptive differential operators	Adaptive DE mutation strategies	Stronger exploitation; faster convergence	Extra parameters
sdABC (Self-adaptive Differential ABC) [44]	**2019**	ABC + self-adaptive differential search	Multiple DE strategies with adaptation	Improved robustness; reduced stagnation	Algorithmic overhead
Bayesian Estimation ABC (BEABC) [45]	**2022**	ABC + Bayesian learning	Probabilistic parameter estimation	High accuracy; strong learning capability	Computationally expensive
Modified ABC for Classification Optimization [46]	**2022**	ABC + classification objective	Task-oriented modification	Improved classification accuracy	Problem-dependent
Modified ABC with DE (mABC-DE) [47]	**2022**	ABC + DE mutation & crossover	Hybrid variation operators	Fast convergence; strong global search	Increased complexity

Table 1 presents the important variants and hybrid approaches of the ABC algorithm. The purpose of the table is to systematically summarize the fundamental approaches proposed in the literature to overcome the known limitations of the classical ABC algorithm, particularly its weak exploitation capability and premature convergence. An examination of Table 1 reveals that early ABC improvements focused primarily on enhancing exploitation capability. Later studies show a shift towards hybridization, adaptive, and learning-based mechanisms. Overall, Table 1 reveals a clear trend in the development of ABC algorithms, moving from structural modifications towards hybrid and adaptive models. This trend provides strong motivation for developing new hybrid ABC-based optimization methods that achieve high performance while avoiding excessive complexity.

Journal Pre-proof

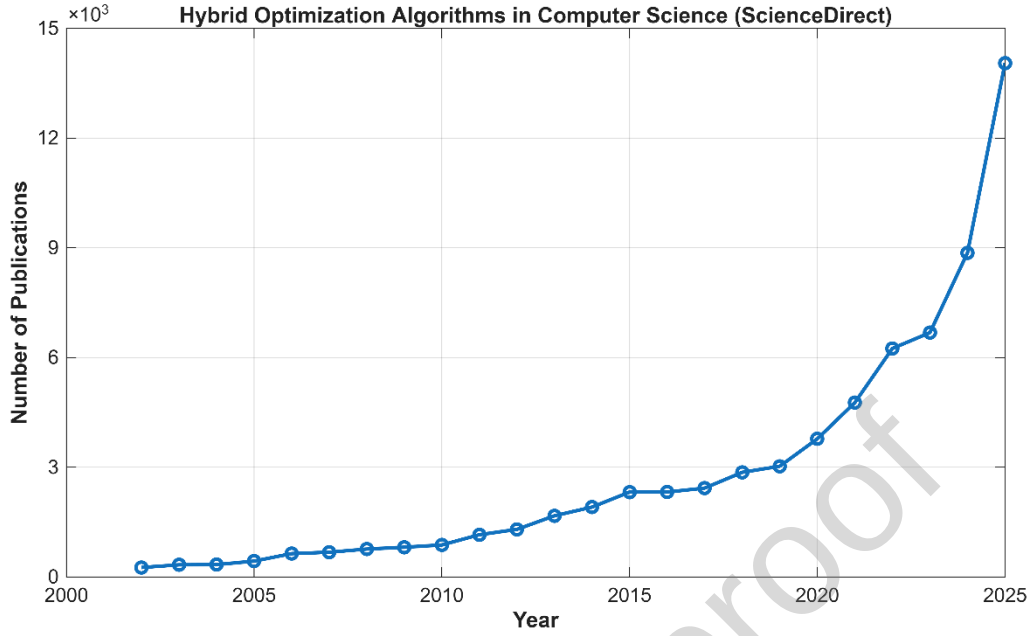


Figure 1. Annual number of publications on hybrid optimization algorithms in Computer Science indexed by ScienceDirect (2002–2025)

Figure 1 shows the number of studies on hybrid optimization algorithms published in the field of Computer Science in the ScienceDirect database between 2002 and 2025. An analysis of the data reveals that the number of publications was quite limited in the early years and showed a gradual increase over the years, with a noticeable and accelerating upward trend emerging since 2019. In particular, the number of studies published in the last year has approximately doubled, indicating that academic interest in hybrid meta-heuristic algorithms is not only increasing but also accelerating. This trend reveals that the superior performance offered by hybrid approaches in complex optimization problems and machine learning applications is gaining increasing acceptance in academic circles.

Despite the numerous meta-heuristic optimization algorithms proposed in recent years, effectively achieving the exploration-exploitation balance in complex and multi-modal problems remains one of the most fundamental and persistent challenges in the field. Although the ABC algorithm is widely accepted due to its simplicity and low parameter requirements, its inherent tendency to explore the solution space in a limited manner and converge early to local optima limits its performance on complex problems. This highlights the need to support the algorithm with a more effective exploration mechanism.

This requirement highlights the potential of new-generation algorithms, particularly those with strong exploration capabilities. In this context, the Starfish Optimization Algorithm (SFOA) [48] demonstrates superior exploration capabilities in multi-modal spaces thanks to its unique trigonometric-based search strategy. This strength of SFOA makes it a natural candidate to complement the weaknesses of ABC. However, SFOA's inadequacy in the exploitation phase, another critical balancing factor, prevents it from being a standalone solution. This weakness of SFOA in local search points to an opportunity to combine its strong discovery capability with an exploitation mechanism that provides accurate and fast convergence.

When examining existing hybridization efforts in the literature, it is evident that algorithms are generally combined superficially, but there is a lack of an integrated framework that systematically addresses the evolutionary nature of search dynamics and the guiding role of past information. This deficiency leads to the persistence of the exploration-exploitation

imbalance even in hybrid structures and prevents algorithms from reaching their full potential. Therefore, there is a need for a new hybrid approach that integrates the strengths of one algorithm to compensate for the weaknesses of another, while also incorporating an intelligent mechanism to manage the evolving nature of the search over time.

This study aims to fill precisely this methodological gap. The robust exploitation structure of the ABC algorithm and the superior exploration power of SFOA are brought together for the first time in this research in a complementary synergy. The proposed Momentum-Guided Search and Starfish Exploration-based Artificial Bee Colony (MSE-ABC) algorithm takes this combination beyond a mechanical addition by enhancing it with a "momentum" mechanism that learns from past search knowledge and dynamically manages the exploration-exploitation transition. Thus, while SFOA's wide-angle exploration effectively scans the solution space in early iterations, ABC's focused exploitation is accelerated in later stages by the directed information provided by momentum, enabling more stable and precise convergence.

The original value of this work stems from the first-time integration of two algorithms into a unified strategy, the idea of providing memory-based guidance to exploration operators, and the introduction of an adaptive framework that dynamically controls search balance. The effectiveness of MSE-ABC has been demonstrated through comprehensive comparative tests and real-world problems, establishing it as a new and powerful option for complex optimization tasks. The main contributions of this study to the literature can be listed as follows:

- A directed and adaptive hybrid optimization model (MSE-ABC) has been proposed, which integrates the ABC and SFOA algorithms under a single umbrella for the first time.
- SFOA's powerful trigonometric exploration operators based on sine and cosine are systematically integrated with a momentum-based mechanism that directly utilizes past search information.
- The exploration-exploitation balance is dynamically and adaptively adjusted based on iterations, thereby developing a directed search strategy that differs from mechanical hybrid approaches based solely on operator combination.
- The proposed MSE-ABC algorithm is designed to reduce early convergence and local optimum getting stuck problems, exhibiting a more stable convergence behavior, especially in complex and multi-modal search spaces.
- The performance of the method has been comprehensively tested on CEC 2022 benchmark functions, three different real-world engineering design problems, and deep neural network hyperparameter optimization scenarios.
- The MSE-ABC algorithm has been compared with contemporary and powerful meta-heuristic algorithms such as success history based differential evolution (SHADE) [49], dream optimization algorithm (DOA) [50], covariance matrix adaptation evolution strategy (CMAES) [51], SFOA, standard ABC, and ABC based on adaptive search strategy and random grouping mechanism ASRGABC [52].
- The statistical significance of the obtained results has been verified using the Wilcoxon signed-rank test.

The remaining sections of the article are organized as follows: Section 2 introduces the basic ABC and SFOA algorithms and presents the detailed structure and mathematical formulation of the proposed MSE-ABC hybrid optimization method. Section 3 contains experimental results, comparisons, and statistical analyses performed on CEC 2022 [53] benchmark functions, engineering design problems, and deep neural network hyperparameter optimization. Section 4 summarizes the overall results of the study and discusses possible future research directions.

Journal Pre-proof

2. Materials and methods

This section provides a detailed examination of the fundamental principles and algorithmic mechanisms of the ABC algorithm and SFOA. The search strategies of both methods, their effects on the exploration–exploitation balance, and their behavior patterns in the solution space are analyzed within a theoretical framework. Furthermore, the step-by-step operation of the algorithms is presented through pseudo-code representations. In this context, the conceptual structure, design motivation, and implementation steps of the MSE-ABC algorithm, a new hybrid approach developed by leveraging the complementary strengths of ABC and SFOA, are explained in detail.

2.1. Artificial Bee Colony Optimization Algorithm

The ABC algorithm is a population-based optimization method that mimics the collective foraging and information sharing behaviors exhibited by honeybees in nature. In the ABC algorithm, each possible solution represents a food source around the beehive, and the quality of this source represents the solution's fitness value. The algorithm consists of three main bee groups: worker bees, scout bees, and observer bees. Worker bees attempt to improve the solution by performing local searches around already discovered food sources. This process is modeled by applying a random but guided perturbation within the neighborhood structure of the current solution.

Observer bees, on the other hand, probabilistically select food sources with higher nectar quantities based on the information shared by worker bees in the dance area. This mechanism enhances exploitation capability by enabling more intensive exploration of promising regions in the solution space. Scout bees, on the other hand, abandon solutions that cannot be improved upon after a certain number of iterations, generating entirely random new points in the solution space. This gives the algorithm a powerful exploration mechanism that prevents it from getting stuck in local minima.

One of the most notable features of the ABC algorithm is that the step size automatically decreases based on progress in the solution space. As solutions get closer to each other, the effect of perturbations diminishes, and the search process becomes more precise. This allows the algorithm to behave adaptively without requiring an additional control parameter. However, the relatively weak local exploitation in the classical ABC structure can cause the convergence speed to be insufficient for some problems. This limitation has paved the way for the emergence of many improved and hybrid approaches based on ABC in the literature.

2.2. Starfish Optimization Algorithm

The SFOA is a modern meta-heuristic optimization method inspired by the trigonometric and symmetrical movements of starfish in response to environmental stimuli. The fundamental motivation behind SFOA is to create a dynamic search behavior that can both perform wide-angle scanning in the solution space and conduct precise searches in promising regions. In the algorithm, each individual represents a sea star located in the solution space, and the movements of these individuals are modeled using sine and cosine functions.

The search process of SFOA is guided by the relative positions of the individuals to the current best solution. Thanks to trigonometric functions, individuals can exhibit nonlinear and periodic movements, jumping to different regions of the solution space. This allows the algorithm to gain strong exploration capabilities in its early stages. In subsequent iterations, as the amplitude of movement gradually decreases, individuals converge around the best solution and shift towards exploitation behavior.

Journal Pre-proof

One of the most significant advantages of SFOA is its ability to establish a smooth and continuous balance between the search and exploitation phases, rather than a sharp transition between them. This structure facilitates avoiding local minima, particularly in multi-modal and complex optimization problems. Furthermore, the simplicity of the algorithm's mathematical structure enhances its applicability and computational efficiency. However, the lack of sufficiently sharp local search capability in the pure SFOA structure for some problems makes hybridization approaches attractive.

2.3. Proposed Momentum-Guided Search and Starfish Exploration-based Artificial Bee Colony (MSE-ABC)

The MSE-ABC algorithm is a new hybrid optimization approach that integrates the phase-based search architecture of the ABC algorithm with the directed and trigonometric motion mechanisms of SFOA under a momentum-supported update strategy. Although the ABC algorithm offers strong local search capabilities through worker, scout, and explorer bee phases, it is known that its global exploration ability weakens over time, especially in complex and multimodal optimization problems. In contrast, SFOA exhibits a broader and more directed exploration behavior in the solution space thanks to its structure inspired by the radial and multi-armed movements of starfish.

In the proposed MSE-ABC algorithm, the basic phase architecture of ABC is preserved; however, the classical neighborhood generation mechanisms have been restructured with directed search components derived from SFOA. Specifically, SFOA's exploration strategies, which involve trigonometric guidance around the global best solution and exploration through randomly selected dimensions, have been integrated into the initial phase of the algorithm to enhance ABC's exploration capability in early iterations. Additionally, the multi-pronged approach used in SFOA's exploitation behavior has been incorporated into ABC's worker bee phase, strengthening its local improvement capacity.

One of the fundamental challenges encountered in the hybridization process is harmoniously combining the different search dynamics of two algorithms. To this end, a momentum-based update mechanism has been defined in the MSE-ABC algorithm. The momentum term, derived from the difference between the global best solutions in successive iterations, provides directional information to the search process and prevents erratic oscillations. This approach ensures that the search direction is maintained in a stable manner, particularly during the exploitation phase.

However, it has been observed that managing the exploration–exploitation balance with fixed parameters leads to suboptimal results in different iteration stages. Therefore, in the MSE-ABC algorithm, an adaptive step size that decreases as the iteration progresses is used to gradually shift the exploration behavior towards exploitation. While the solution space is broadly scanned with large steps in the initial phase, smaller steps are used in subsequent iterations to perform precise local searches. Thanks to this dynamic structure, both the algorithm's global search capability and convergence speed have been significantly increased.

In the MSE-ABC algorithm, each candidate solution is defined as an nD-dimensional vector as in Eq. (1).

$$\mathrm{X_i} = \left[\mathrm{x}_{i,1}, \mathrm{x}_{\mathrm{i},2}, \dots, \mathrm{x}_{\mathrm{i,nD}}\right] \tag{1}$$

Here, X_i is the nD-dimensional decision variable vector representing the ith candidate solution. $x_{\{i,j\}}$, corresponds to the jth decision variable component of the ith solution, and nD denotes the dimension of the optimization problem.

The initial population is randomly distributed within the feasible search space using a uniform distribution, as expressed in Eq. (2).

$$x_{i,j} = lb_j + rand(0,1) \times \left(ub_j - lb_j \right) \quad (2)$$

Here, lb_j and ub_j denote the lower and upper bounds of the j-th decision variable, respectively. $rand(0,1)$ is a uniformly distributed random number in the interval [0,1]. This formulation ensures a diverse and unbiased initial population.

Once the population is initialized, the fitness value of each candidate solution is evaluated using the objective function$f(\cdot)$, as shown in Eq. (3).

$$Fitness_i = f(X_i) \quad (3)$$

Here, $Fitness_i$ represents the fitness value of the ith solution, and $f(\cdot)$ denotes the objective function to be minimized.

At the initialization stage, the global best solution is determined based on the minimum fitness value among all individuals in the population, as defined in Eqs. (4) and (5).

$$BestFit(0) = \min_{i=1,2\ldots,Npop} Fitness_i \quad (4)$$

Here, $BestFit(0)$ denotes the minimum fitness value obtained at the initial iteration, $BestPos(0)$ represents the corresponding decision vector, $Fitness_i$ is the fitness value of the ith candidate solution, and $Npop$ indicates the population size.

These quantities define the reference solution that guides the subsequent search process.

$$BestPos(0) = argmin_{X_i} Fitness_i \quad (5)$$

Here, $BestFit(0)$ denotes the minimum fitness value obtained at the initial iteration, and $BestPos(0)$ represents the corresponding decision vector. This global best solution serves as a reference point for guiding the subsequent search process.

To introduce directional memory into the search process, MSE-ABC explicitly defines a momentum term derived from successive global best solutions, which is calculated using Eq. (6).

$$Momentum(t) = BestPos(t) - BestPos(t-1) \quad (6)$$

Here, $Momentum(t)$ represents the displacement vector between the global best positions obtained at iterations t and $t-1$. This term captures the dominant search direction in the solution space.

The momentum contribution is scaled by a control parameter to regulate its influence on the search dynamics, as defined in Eq. (7).

$$M(t) = \beta \times Momentum(t) \quad (7)$$

Here, $\beta \in (0,1)$ is the momentum factor controlling the strength of directional persistence. Higher values of β emphasize exploitation along the dominant direction, whereas lower values increase exploratory behavior.

In MSE-ABC, an adaptive step size mechanism is employed to dynamically balance exploration and exploitation during the optimization process, as formulated in Eq. (8).

$$step(t) = step_{init} \times \left(\frac{step_{final}}{step_{init}} \right)^{\left\{ \frac{t}{MaxIt} \right\}} \quad (8)$$

Journal Pre-proof

Here, $step_{init}$ and $step_{final}$ denote the initial and final step sizes, respectively, t is the current iteration index, and $MaxIt$ represents the maximum number of iterations. This exponentially decreasing function enables wide-ranging exploration in early iterations and fine-grained local search in later stages.

At each iteration, MSE-ABC probabilistically determines whether a candidate solution performs exploration or exploitation according to Eq. (9).

$$S_i(t) = \begin{cases} \varepsilon, & r_i(t) < GP \\ X, & r_i(t) \geq GP \end{cases} \quad (9)$$

Here, $S_i(t)$ denotes the search behavior assigned to the ith solution at iteration t, where ε and X represent exploration and exploitation modes, respectively. The variable $r_i(t) \sim U(0,1)$ is a uniformly distributed random number, and $GP \in (0,1)$ is the exploration probability controlling the transition between global exploration and local exploitation.

When exploration is selected, the candidate solution is updated toward the global best position while incorporating momentum information as expressed in Eq. (10).

$$x_{i,j}^{new} = x_{i,j} + step(t) \times \left[r_1 \times \left(BestPos_j - x_{i,j} \right) + \beta \times Momentum_j(t) \right] \quad (10)$$

Here, $r_1 \in (0,1)$ is a random coefficient, $BestPosj$ denotes the jth component of the global best solution, and $Momentum_j(t)$ represents the jth component of the momentum vector. This update encourages directed global exploration with temporal consistency.

In the exploitation phase, the search is intensified around promising regions using relative positional information, as described in Eq. (11).

$$X_i^{new} = X_i + r_2 \times (BestPos - X_k) \quad (11)$$

Here, $r_2 \in (0,1)$ is a random scaling factor, and X_k denotes a randomly selected solution from the population. This mechanism enhances local exploitation around high-quality solutions.

Following the phase, the employed bee phase of the ABC algorithm is executed with momentum support, as formulated in Eq. (12).

$$x_{i,j}^{new} = x_{i,j} + \left(x_{i,j} - x_{k,j} \right) \times \varphi + step(t) \times \beta \times Momentum_j(t) \quad (12)$$

Here, $\varphi \in [-1,1]$ is a random perturbation coefficient. This formulation extends the classical ABC employed bee update rule by incorporating adaptive step size and momentum information, thereby improving convergence speed and search stability.

In the onlooker bee phase, candidate solutions are selected based on their fitness-dependent probabilities, as defined in Eq. (13).

$$P_i = 0.9 \times \frac{\left[\frac{1}{(1 + Fitness_i)} \right]}{\max \left[\frac{1}{(1 + Fitness)} \right]} + 0.1 \quad (13)$$

Here, P_i denotes the probability of selecting the ith solution. This normalized formulation favors high-quality solutions while maintaining population diversity.

To prevent stagnation, solutions that fail to improve for a predefined number of trials are abandoned and replaced, as expressed in Eq. (14).

$$X_i = lb + rand \times (ub - lb) \quad (14)$$

Journal Pre-proof

This equation represents the scout bee mechanism, where abandoned solutions are reinitialized randomly within the search space, thereby enhancing exploration and avoiding premature convergence.

At the end of each iteration, the global best solution is updated as described in Eq. (15).

$$BestFit(t) = \min(\ Fitness_i(t)) \quad (15)$$

$$BestPos(t) \ = \ argmin_{X_i} \ (\ Fitness_i(t)\ )$$

These expressions update the best fitness value and its corresponding solution vector obtained up to iteration t.

Algorithm 1 presents the pseudocode of the proposed MSE-ABC algorithm. The proposed MSE-ABC algorithm integrates the guided exploration capability of the Starfish Optimization Algorithm with the strong local exploitation mechanism of the Artificial Bee Colony algorithm. At the beginning, a population of candidate solutions is randomly initialized and evaluated. During each iteration, a starfish-inspired phase is first employed to perform momentum-guided exploration or global-best-oriented exploitation, which enhances population diversity and prevents premature convergence. Subsequently, the employed and onlooker bee phases of the ABC algorithm are applied to intensify the search around promising regions through fitness-based local exploitation. Finally, a scout bee mechanism is used to reinitialize stagnated solutions, maintaining diversity throughout the optimization process. The global best solution is updated iteratively, and the convergence behavior is recorded until the termination criterion is satisfied.

By combining the complementary strengths of SFOA and ABC, the proposed MSE-ABC algorithm achieves a balanced exploration–exploitation trade-off. The momentum-guided starfish phase accelerates convergence toward high-quality regions, while the ABC-based local search phases refine solutions and improve stability, particularly in complex and multimodal optimization problems.

Algorithm 1. Pseudocode of the Proposed MSE-ABC (Momentum-Guided Search and Starfish Exploration-based Artificial Bee Colony) Algorithm

Algorithm 1

MSE-ABC: Momentum-Guided Search and Starfish Exploration-based Artificial Bee Colony

Input: f(·), lb, ub, nD, N_{pop}, MaxIt, β (momentum_factor), GP, $step_{init}$, $step_{final}$, limit, fobj
Output: BestPos, BestFit, Curve
INITIALISATION
for i = 1 **to** N_{pop} **do**
  for j = 1 **to** nD **do**
    $x_{i,j} \leftarrow lb_j + rand(0,1) \times (ub_j - lb_j)$ **[Eq. 2]**
  end for
  $Fitness_i \leftarrow fobj(X_i)$ **[Eq. 3]**
  $trial_i \leftarrow 0$ *// stagnation counter for scout bee*
end for
$BestFit(0) \leftarrow min_i(Fitness_i)$ **[Eq. 4]**
$BestPos(0) \leftarrow argmin_{Xi}(Fitness_i)$ **[Eq. 5]**
$BestPos_{prev} \leftarrow BestPos(0)$ *// store for momentum computation*
MAIN LOOP
for t = 1 **to** MaxIt **do**
  // Adaptive step size (exponential decay)
  $step(t) \leftarrow step_{init} \times (step_{final} / step_{init})^{\wedge}(t / MaxIt)$ **[Eq. 8]**
  // Momentum vector from successive global bests
  $Momentum(t) \leftarrow BestPos(t-1) - BestPos_{prev}$ **[Eq. 6]**
  $M(t) \leftarrow \beta \times Momentum(t)$ **[Eq. 7]**
  $BestPos_{prev} \leftarrow BestPos(t-1)$ *// update for next iteration*
// ═══════ STARFISH PHASE ═══════

Journal Pre-proof

```
18 for i = 1 to N_pop do
    19 r_i(t) ← rand(0,1)
    20 if r_i(t) < GP then    // GP = 0.5; exploration branch
        21 j ← randi(nD)    // single random dimension
        22 r_1 ← rand(0,1)
        23 x^new_i,j ← x_i,j + step(t) × [r_1 × (BestPos_j − x_i,j) + β × Momentum_j(t)]    [Eq. 10]
    24 else    // exploitation branch
        25 k ← randi(N_pop), k ≠ i
        26 r_2 ← rand(0,1)
        27 X^new_i ← X_i + r_2 × (BestPos − X_k)    [Eq. 11]
    28 end if
    29 // Boundary control: clipping to [lb, ub]
    30 for j = 1 to nD do
        31 if x^new_i,j < lb_j or x^new_i,j > ub_j then
            32 x^new_i,j ← lb_j + rand(0,1) × (ub_j − lb_j)
        33 end if
    34 end for
    35 // Greedy selection
    36 if fobj(X^new_i) < Fitness_i then
        37 X_i ← X^new_i;  Fitness_i ← fobj(X^new_i);  trial_i ← 0
    38 else
        39 trial_i ← trial_i + 1
    40 end if
41 end for
// ═══════════ ABC: EMPLOYED BEE PHASE ═══════════
42 for i = 1 to FoodNumber do
    43 k ← randi(FoodNumber), k ≠ i
    44 j ← randi(nD)    // single random dimension
    45 φ ← (rand(0,1) − 0.5) × 2    // φ ∈ [−1, 1]
    46 x^new_i,j ← x_i,j + (x_i,j − x_k,j) × φ + step(t) × β × Momentum_j(t)    [Eq. 12]
    47 // Boundary control (same rule as lines 30–34)
    48 // Greedy selection
    49 if fobj(X^new_i) < Fitness_i then
        50 X_i ← X^new_i;  Fitness_i ← fobj(X^new_i);  trial_i ← 0
    51 else
        52 trial_i ← trial_i + 1
    53 end if
54 end for
// ═══════════ ABC: ONLOOKER BEE PHASE ═══════════
55 for i = 1 to FoodNumber do
    56 P_i ← 0.9 × [1 / (1 + Fitness_i)] / max[1 / (1 + Fitness)] + 0.1    [Eq. 13]
57 end for
58 t_sel ← 0;  i ← 1
59 while t_sel < FoodNumber do
    60 if rand(0,1) < P_i then    // roulette-wheel acceptance
        61 k ← randi(FoodNumber), k ≠ i
        62 j ← randi(nD)
        63 φ ← (rand(0,1) − 0.5) × 2
        64 x^new_i,j ← x_i,j + (x_i,j − x_k,j) × φ    [Eq. 12]
        65 // Boundary control (same rule as lines 30–34)
        66 if fobj(X^new_i) < Fitness_i then
            67 X_i ← X^new_i;  Fitness_i ← fobj(X^new_i);  trial_i ← 0
        68 else
            69 trial_i ← trial_i + 1
        70 end if
        71 t_sel ← t_sel + 1
    72 end if
    73 i ← (i mod FoodNumber) + 1    // cycle through food sources
74 end while
// ═══════════ SCOUT BEE PHASE ═══════════
```

Journal Pre-proof

```
    idx ← argmax_i(trial_i)
    if trial_idx ≥ limit then    // abandonment condition (limit = 100)
        for j = 1 to nD do
            x_idx,j ← lb_j + rand(0,1) × (ub_j − lb_j)    [Eq. 14]
        end for
        Fitness_idx ← fobj(X_idx)    [Eq. 3]
        trial_idx ← 0    // reset counter after re-initialisation
    end if
    // ═══════════ GLOBAL BEST UPDATE ═══════════
    [minFit, idx] ← min_i(Fitness_i)
    if minFit < BestFit then
        BestFit ← minFit;  BestPos ← X_idx    [Eq. 15]
    end if
    Curve(t) ← BestFit    // store convergence value
end for
return BestPos, BestFit, Curve
```

Note: The code will be shared on GitHub at **github/hakbulut60** for the benefit of the research community.

3. Experimental Results

In this section, the effectiveness and robustness of the proposed MSE-ABC algorithm have been evaluated through a comprehensive experimental analysis. The experimental study first utilized standard CEC 2022 benchmark functions to demonstrate the algorithm's general search capability and convergence behavior in complex and multimodal optimization problems. In the second stage, experiments were conducted on the Welded Beam, Pressure Vessel, Tension Spring, Speed Reducer, and Three Bar Truss engineering design problems [54] to demonstrate the practical applicability and performance of the proposed method on real-world problems. Finally, deep learning-based hyperparameter tuning scenarios were addressed on the CIFAR-10 datasets.

All experiments were conducted in the MATLAB environment on a computer equipped with a Windows 10 Pro operating system, a 12th generation Intel® Core™ i5-12400 (2.50 GHz) processor, and 16 GB of RAM. All comparative experiments were conducted under common parameters to ensure a fair and reproducible evaluation; each algorithm was tested with 30 independent runs. In the experiments, the population size was set to 30, the maximum iteration count to 1000, and the algorithms included in the comparison were run with the default parameter values recommended in the literature. The proposed MSE-ABC algorithm was compared with current and widely used meta-heuristic algorithms such as ABC, ASRGABC, SFOA, SHADE, DOA, and CMAES. The ASRGABC algorithm was selected as the reference method because it is one of the most powerful variants of the ABC algorithm proposed in 2022 and has demonstrated superior performance in comparisons with other ABC derivatives in the literature. The other algorithms were selected because they have been recently proposed or are widely used in the literature. Performance evaluation was conducted based on the mean best value, standard deviation, computation time, convergence curves, and statistical significance tests (Wilcoxon signed-rank test and Friedman test).

3.1. Performance Evaluation of MSE-ABC on the low-dimensional CEC 2022 Functions

In this subsection, the overall performance of the proposed MSE-ABC algorithm in multi-modal optimization problems has been comprehensively evaluated using the CEC 2022 benchmark functions. The CEC 2022 test set, consisting of single-modal, simple multi-modal, hybrid, and composite functions, provides a challenging test environment that is widely accepted for balancing the exploration and exploitation capabilities of an optimization algorithm. The search space bounds for all CEC 2022 functions are defined in the range [−100, 100] as suggested in the literature, and the problem size is fixed at 20.

Journal Pre-proof

Table 2. Performance comparison of the algorithms on CEC 2022 benchmark functions.

Function	Algorithm	Best	Mean	SD	Time Mean(s)	P Value	Effect Size	Significant	Winner
CEC1	ABC	29560.8422	38959.3109	5860.3794	0.4193	1.7344E-06	0.9922	YES	MSE-ABC
	ASRGABC	21270.6594	33879.7357	6928.9982	0.0739	1.7344E-06	0.9911	YES	
	SFOA	300.6294	325.1797	25.7670	0.0387	1.7344E-06	0.0703	YES	
	SHADE	20200.0221	42593.5540	9872.5933	0.3292	1.7344E-06	0.9929	YES	
	DOA	378.2491	2908.1311	2088.4087	0.0437	1.7344E-06	0.8960	YES	
	CMAES	10367.0743	25610.3060	11085.7794	0.1242	1.7344E-06	0.9882	YES	
	MSEABC	300.0016	**302.3045**	7.2961	0.0664	-		-	
CEC2	ABC	449.0906	449.1047	0.0139	0.4226	1.7344E-06	0.0307	YES	MSE-ABC
	ASRGABC	435.0379	447.9226	2.9034	0.0720	4.07151E-05	0.0282	YES	
	SFOA	404.1468	446.4041	16.9163	0.0377	0.031603382	0.0249	YES	
	SHADE	445.2198	450.9459	5.2190	0.3288	1.7344E-06	0.0347	YES	
	DOA	405.1705	442.7604	13.1148	0.0448	0.015658483	0.0168	YES	
	CMAES	444.9708	447.7610	1.9834	0.1220	0.001197338	0.0278	YES	
	MSEABC	400.0000	**435.3032**	21.3905	0.0661	-	-	-	
CEC3	ABC	600.0480	600.2532	0.1226	0.6032	1.7344E-06	0.0004	YES	MSEABC
	ASRGABC	600.0000	600.0003	0.0015	0.1480	0.018518975	0.0000	NO	
	SFOA	601.9006	607.0188	3.2830	0.1190	1.7344E-06	0.0144	YES	
	SHADE	600.0000	600.0043	0.0213	0.4139	0.002765274	0.0000	NO	
	DOA	600.0105	600.0269	0.0183	0.1287	1.7344E-06	0.0000	YES	
	CMAES	600.0000	600.0043	0.0232	0.2019	0.005319684	0.0000	YES	
	MSEABC	600.0000	**600.0001**	0.0001	0.2212	-	-	-	
CEC4	ABC	906.5135	926.9227	7.9052	0.4639	1.7344E-06	0.0681	YES	CMAES
	ASRGABC	853.0042	883.1080	14.7437	0.0962	0.000771217	0.0219	YES	
	SFOA	822.3624	875.3199	23.7345	0.0609	0.047161747	0.0132	YES	
	SHADE	831.3686	854.1242	11.7210	0.3518	0.028485956	0.0112	YES	
	DOA	829.8491	853.1770	14.4363	0.0668	0.035008957	0.0123	YES	
	CMAES	806.9647	**814.8580**	4.7747	0.1481	1.7344E-06	0.0566	YES	
	MSEABC	828.8552	863.7586	19.5963	0.1132	-	-	-	
CEC5	ABC	902.1856	926.9594	19.2001	0.4746	1.7344E-06	0.4355	YES	CMAES
	ASRGABC	1554.7776	2195.2087	336.4505	0.0992	3.18168E-06	0.2520	YES	
	SFOA	912.0958	1326.4995	470.6188	0.0626	0.004114031	0.1922	YES	
	SHADE	1061.3874	1320.0283	213.3860	0.3543	0.000715703	0.1961	YES	
	DOA	946.1703	1282.0351	335.7614	0.0677	0.000715703	0.2193	YES	
	CMAES	900.0000	**900.0000**	0.0000	0.1512	1.7344E-06	0.4519	YES	
	MSEABC	1152.6300	1642.1033	325.9786	0.1155	-	-	-	
CEC6	ABC	477288.8810	3585220.9318	1755488.7529	0.4282	1.7344E-06	0.9995	YES	MSEABC

Journal Pre-proof

	ASRGABC	3099.5136	139083.5565	206507.3948	0.0783	1.7344E-06	0.9867	YES	
	SFOA	1894.5503	3259.7724	4528.9512	0.0429	2.12664E-06	0.4320	YES	
	SHADE	4896.7145	157262.6500	155531.3279	0.3338	1.7344E-06	0.9882	YES	
	DOA	1825.8205	3401.1701	2326.5377	0.0488	2.60333E-06	0.4556	YES	
	CMAES	1909.9953	8579.3243	5740.6128	0.1270	1.7344E-06	0.7842	YES	
	MSEABC	1803.2288	**1851.4390**	36.3020	0.0760	-	-	-	
CEC7	ABC	2081.5650	2116.2709	11.6535	0.6532	1.7344E-06	0.0388	YES	MSEABC
	ASRGABC	2022.9817	2039.9697	7.6210	0.1764	0.025637124	0.0029	YES	
	SFOA	2035.1513	2089.1688	27.6614	0.1387	1.92092E-06	0.0264	YES	
	SHADE	2024.5180	2038.6141	9.1028	0.4456	0.075213312	0.0000	NO	
	DOA	2021.1286	2035.8091	16.1436	0.1470	0.734325291	0.0000	NO	
	CMAES	2026.6048	2049.8058	13.8859	0.2300	5.30699E-05	0.0077	YES	
	MSEABC	2021.4588	**2034.0688**	10.0305	0.2681	-	-	-	
CEC8	ABC	2235.9918	2248.9938	4.5873	0.6658	1.7344E-06	0.0122	YES	MSEABC
	ASRGABC	2224.0854	2226.3152	1.4878	0.1911	1.7344E-06	0.0021	YES	
	SFOA	2228.1674	2241.5809	21.3234	0.1531	1.7344E-06	0.0089	YES	
	SHADE	2223.0151	2224.4449	0.8883	0.4473	1.7344E-06	0.0013	YES	
	DOA	2220.6138	2225.2045	21.7159	0.1585	0.065641143	0.0000	NO	
	CMAES	2222.8712	2263.9043	52.0678	0.2416	1.7344E-06	0.0187	YES	
	MSEABC	2220.8399	**2221.5873**	0.5407	0.2971	-	-	-	
CEC9	ABC	2480.8115	2480.9634	0.1656	0.6334	1.7344E-06	0.0001	YES	MSEABC
	ASRGABC	2480.7840	2481.1505	0.5582	0.1770	1.7344E-06	0.0001	YES	
	SFOA	2480.7813	2480.7824	0.0042	0.1412	1.7344E-06	0.0000	YES	
	SHADE	2480.7815	2481.3476	0.8149	0.4334	1.7344E-06	0.0002	YES	
	DOA	2480.7815	2481.1074	0.4799	0.1547	1.7344E-06	0.0001	YES	
	CMAES	2480.8183	2480.8627	0.0278	0.2291	1.7344E-06	0.0000	YES	
	MSEABC	2480.7813	**2480.7813**	0.0000	0.2721	-	-	-	
CEC10	ABC	2506.4545	2526.7185	15.6361	0.5795	0.000615641	0.0237	YES	ASRGABC
	ASRGABC	2400.1874	**2417.8506**	35.3259	0.1465	0.000331726	0.0199	YES	
	SFOA	2500.6237	3718.0449	1475.2751	0.1123	9.31566E-06	0.3365	YES	
	SHADE	2400.0625	2430.9010	57.9033	0.4054	0.014795424	0.0146	YES	
	DOA	2400.3469	2450.7043	55.2889	0.1195	0.673279807	0.0000	NO	
	CMAES	2448.3427	2612.0387	174.0866	0.2014	0.000135948	0.0556	YES	
	MSEABC	2402.0205	2466.9317	71.2790	0.2163	-	-	-	
CEC11	ABC	2900.0917	2901.5252	3.0776	0.7309	0.008729668	0.0407	YES	MSEABC
	ASRGABC	2603.6193	2885.0325	100.0151	0.2196	0.036826128	0.0352	YES	
	SFOA	2901.4211	3003.1668	99.6251	0.1855	4.28569E-06	0.0732	YES	
	SHADE	2900.0000	2975.1419	98.1528	0.4795	0.001286631	0.0645	YES	

Journal Pre-proof

	DOA	2600.7132	2850.6470	113.5986	0.1902	0.110925665	0.0000	NO	
	CMAES	2900.0000	2906.6667	25.3708	0.2739	0.020671114	0.0424	YES	
	MSEABC	2600.0000	**2783.3600**	189.4979	0.3592	-	-	-	
CEC12	ABC	2939.8784	2949.5028	3.8802	0.7555	0.000962659	0.0018	YES	CMAES
	ASRGABC	2940.0363	2949.9607	4.7120	0.2389	0.000830707	0.0020	YES	
	SFOA	2938.4962	2961.7442	31.0225	0.2020	0.000189097	0.0059	YES	
	SHADE	2937.0831	2944.5037	5.1128	0.4923	0.958990172	0.0000	NO	
	DOA	2934.0085	2945.0136	6.6894	0.2164	0.585711569	0.0000	NO	
	CMAES	2930.8492	**2937.8137**	5.0613	0.2902	0.000615641	0.0021	YES	
	MSEABC	2933.0101	2944.1318	6.5805	0.3932	-	-	-	
MSEABC 8 winner. CMAES 3 winner. ASRGABC 1 winner									
SD: Standard Deviation									

Table 2 presents the comparative performance results of seven different metaheuristic algorithms on the CEC 2022 benchmark functions. The results include the best value (Best), mean value (Mean), standard deviation (SD), mean execution time (Time Mean), Wilcoxon signed-rank test p-value, effect size, statistical significance, and winner algorithm information obtained from 30 independent runs.

The Friedman test, applied to determine the overall performance differences among the algorithms, revealed the presence of statistically significant differences between the groups. Examination of the Wilcoxon signed-rank test results used for pairwise comparisons showed that statistically significant differences at the $p < 0.05$ level were obtained against at least one algorithm in 10 out of 12 test functions (83.3%). This finding confirms that the search dynamics of the examined metaheuristic algorithms differ markedly from one another and that the problem structure plays a decisive role in algorithm performance.

The distribution of winner algorithms was determined based on the mean value (Mean), and the proposed MSE-ABC algorithm demonstrated its overall superiority by achieving the lowest mean value in 8 out of 12 functions (66.7%). This was followed by CMA-ES, which achieved the winner position in 3 functions (25.0%), namely CEC4, CEC5, and CEC12, while ASRGABC reached the winner position in 1 function (8.3%), namely CEC10. The ABC, SFOA, SHADE, and DOA algorithms did not achieve the lowest mean value in any function. The average rank analysis also supports this result, with MSE-ABC emerging as the algorithm with the lowest average rank across the entire function set.

When examined on a function basis, in CEC1, MSE-ABC exhibited a distinct superiority over its competitors in terms of both solution quality and stability, with a mean value of 302.30 and a standard deviation of 7.30. It was observed that the compared algorithms (ABC: 38,959.31; SHADE: 42,593.55; ASRGABC: 33,879.74) produced substantially higher mean values, and all of them lagged significantly behind at the $p < 0.05$ level. The effect size values ranging between 0.89 and 0.99 indicate that this superiority is also highly meaningful in practical terms. In CEC2, MSE-ABC maintained the winner position with a mean value of 435.30; however, in this function, the standard deviation of MSE-ABC (21.39) was considerably higher than those of its competitors such as SHADE (5.22), CMA-ES (1.98), and ASRGABC (2.90). This situation indicates that although MSE-ABC was superior in terms of average performance in this function, it remained relatively limited with respect to solution stability.

Journal Pre-proof

In the CEC3 and CEC9 functions, MSE-ABC demonstrated perfect stability with standard deviation values of 0.0001 and 0.0000, respectively. While all competing algorithms lagged statistically significantly behind in CEC9 ($p = 1.73\times10^{-6}$), ASRGABC and SHADE exhibited statistically equivalent performance in CEC3 and were evaluated as a "Tie." The findings in these two functions reveal that MSE-ABC possesses a stable convergence capability in smooth and low-modal function spaces. In CEC4, however, CMA-ES became the winner; with a mean value of 814.86 and a standard deviation of 4.77, CMA-ES was the only algorithm in this function that simultaneously provided both the lowest mean value and the highest stability. MSE-ABC produced a mean value of 863.76 in this function and lagged significantly behind statistically ($p = 1.73\times10^{-6}$). This finding suggests that the covariance adaptation mechanism of CMA-ES may provide a structural advantage in certain unimodal or weakly multimodal structures.

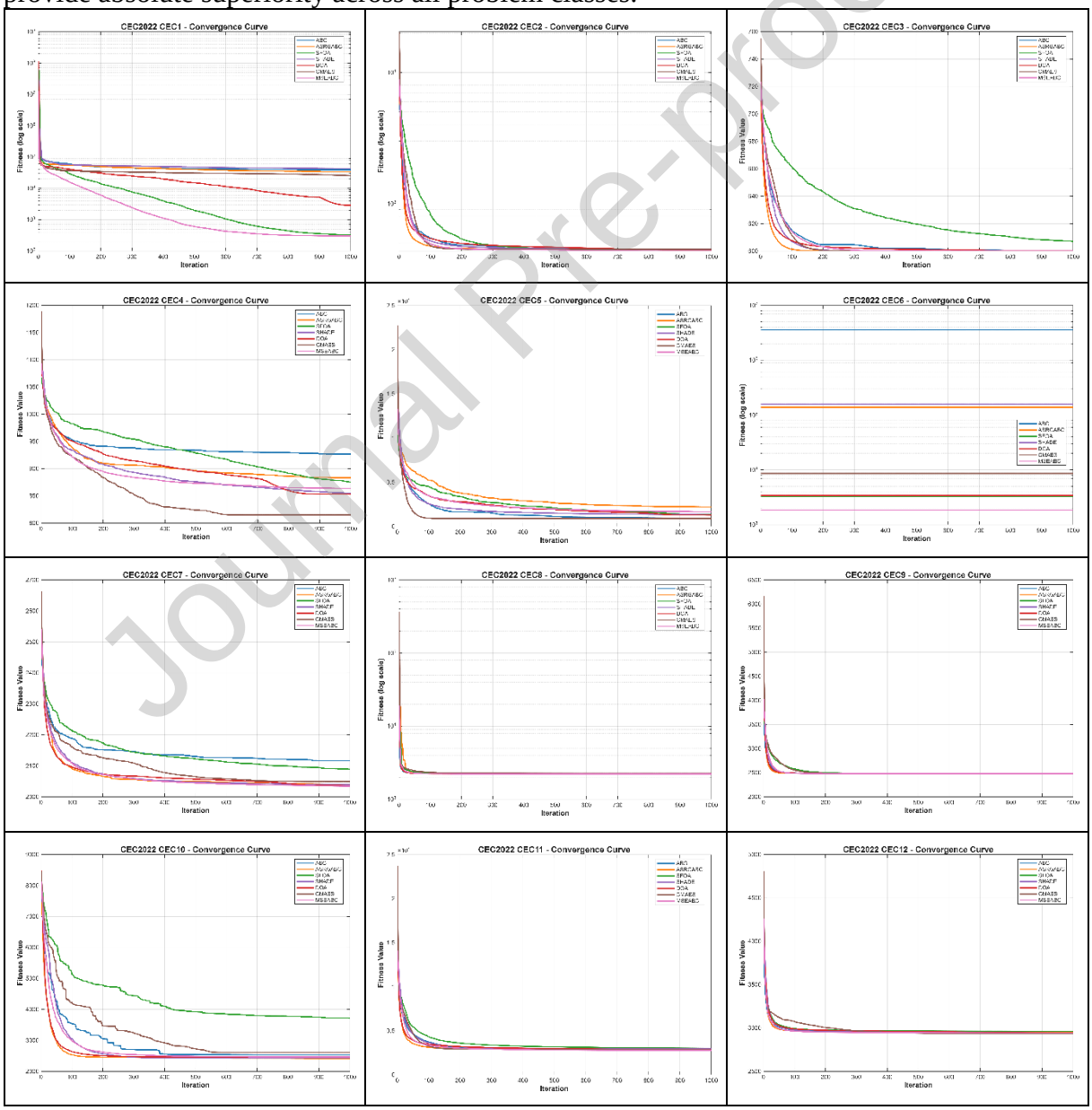
CEC5 stands out as the function in which MSE-ABC exhibited its weakest performance. In this function, MSE-ABC remained behind all competitors except ASRGABC, with a mean value of 1,642.10 and a standard deviation of 325.98. The winner, CMA-ES, produced both the highest-quality and the most stable solution with a mean value of 900.00 and a zero standard deviation. ABC (926.96) also demonstrated competitive performance in this function. The structural characteristics of CEC5—possibly strong local optimum traps or irregular search space geometry—negatively affected the perturbation mechanism of MSE-ABC and revealed that the algorithm is relatively fragile against such function structures. CEC6 emerged as the most difficult function in terms of evaluation; ABC exhibited extremely poor performance in this function with a mean value of 3,585,220.93, while SHADE (157,262.65) and ASRGABC (139,083.56) also produced inconsistent results due to high variance. MSE-ABC, on the other hand, achieved the most stable and highest-quality solution among all competitors with a mean value of 1,851.44 and a standard deviation of only 36.30. The satisfaction of the $p < 0.05$ significance threshold in all comparisons and the effect size values varying between 0.43 and 0.99 confirm that this superiority is supported by a strong statistical foundation.

In the CEC7 and CEC8 functions, MSE-ABC was in the winner position; however, in CEC7, SHADE ($p = 0.075$) and DOA ($p = 0.734$), and in CEC8, DOA ($p = 0.066$), failed to produce statistically significant differences and resulted in a "Tie" decision. This situation demonstrates that even in functions where MSE-ABC outperformed in terms of average performance, certain competing algorithms could still remain within the boundaries of statistical equivalence, empirically supporting that no single algorithm can achieve absolute superiority across the entire function space. A remarkable result was obtained in CEC10: ASRGABC became the winner in this function by producing the lowest mean value of 2,417.85. MSE-ABC exhibited a higher variance in this function with a mean value of 2,466.93 and a standard deviation of 71.28; thus, it is understood that the adaptive search strategy of ASRGABC can improve local search quality in certain multimodal structures. On the other hand, DOA was considered statistically equivalent in this function ($p = 0.673$). In CEC11, MSE-ABC was the winner with a mean value of 2,783.36; however, the standard deviation of 189.50 in this function corresponds to the highest variability recorded for MSE-ABC across the entire test set. This situation indicates that the algorithm had difficulty achieving consistent convergence in the rugged search landscape of CEC11 and that the performance variability between runs should be evaluated carefully. In CEC12, CMA-ES became the winner; while CMA-ES produced a mean value of 2,937.81, MSE-ABC lagged significantly behind statistically with a mean value of 2,944.13 ($p = 0.001$). SHADE and DOA were evaluated as statistically equivalent to MSE-ABC in this function.

When the average execution times are examined, it is observed that the ABC algorithm resulted in the highest computational cost across all functions (CEC1: 0.419 s, CEC11: 0.731 s, CEC12:

0.756 s). SHADE also exhibited relatively high execution time values in composite functions. In contrast, MSE-ABC achieved competitive execution times in the range of 0.066–0.399 s in most functions. Overall, despite containing additional search mechanisms, MSE-ABC does not impose excessive computational burden and provides a suitable balance between time and quality for practical applications.

When the obtained findings are evaluated as a whole, it is observed that the MSE-ABC algorithm exhibits a statistically strong and consistent performance profile on the CEC 2022 benchmark set. Achieving the winner position in 8 out of 12 functions, the significant superiority obtained over competing algorithms in the Wilcoxon test, and the competitive computational times confirm that the proposed method is an effective optimization tool in terms of both solution quality and practical applicability. Nevertheless, the fact that CMA-ES reached the winner position in three functions (CEC4, CEC5, CEC12) and ASRGABC in one function (CEC10) emphasizes the importance of problem-specific algorithm selection and empirically supports the "No Free Lunch" theorem, which states that no single metaheuristic method can provide absolute superiority across all problem classes.



Journal Pre-proof

Figure 2. Convergence behavior comparison of the proposed and competing algorithms on CEC-2022 benchmark functions

Figure 2 presents the convergence behaviors of the proposed MSE-ABC algorithm and the comparative algorithms on the CEC-2022 benchmark functions. The subfigures are arranged sequentially from CEC1 to CEC12 and illustrate how the algorithms improve the objective function values throughout the iterations. The CEC-2022 test suite consists of unimodal, basic, hybrid, and composition functions with different difficulty levels. This structure enables the evaluation of not only convergence speed, but also exploration–exploitation balance, the ability to avoid local minima, robustness in complex and multimodal search spaces, and stability in complex fitness landscapes. Overall, the convergence curves indicate that the MSE-ABC algorithm exhibits a stable and sustainable optimization behavior across different problem categories.

CEC1 is a unimodal problem based on the shifted and rotated Zakharov function. For such functions, the primary expectation is that the algorithm should rapidly converge toward the optimum region through strong exploitation capability. The convergence curves demonstrate that MSE-ABC achieves a faster reduction from the early iterations compared to the competing methods and maintains a more stable convergence behavior throughout the optimization process. In particular, ABC and SHADE remain at higher error levels, while SFOA and DOA lose their convergence speed after certain iterations. In contrast, MSE-ABC approaches the optimum region in a controlled and stable manner, demonstrating its strong exploitation capability.

CEC2 is based on the rotated Rosenbrock function, which contains strong inter-variable dependency and a narrow valley structure. In such problems, algorithms generally exhibit rapid improvement during the initial iterations but struggle to progress along the optimum valley. The convergence curves show that all algorithms achieve a rapid decline at the beginning; however, MSE-ABC reaches lower error levels in a shorter time. While ASRGABC and CMAES slow down after specific iterations, MSE-ABC maintains a more balanced convergence process. This behavior indicates that the proposed method possesses effective guidance mechanisms in highly correlated search spaces.

CEC3 is derived from the expanded Schaffer's F6 function, which contains intensive multimodality and highly complex fitness landscapes around the optimum region. In such functions, the ability of algorithms to avoid local minima and move toward the global optimum is critically important. The convergence curves indicate that although several algorithms can generate near-optimal solutions, MSE-ABC preserves its early convergence advantage. In particular, SFOA follows a slower improvement process, while some algorithms exhibit plateau behavior. In contrast, MSE-ABC demonstrates a more balanced convergence profile and performs a more effective search within the multimodal landscape.

CEC4 is based on the discontinuous and rotated Rastrigin function, which possesses a highly multimodal and complex search space. Discontinuities and numerous local minima tend to cause premature convergence in optimization algorithms. The convergence curves show that MSE-ABC differentiates itself from the competing algorithms from the early iterations and follows a more balanced improvement process. While ABC and SFOA lose convergence speed in the later iterations, MSE-ABC maintains a more sustainable decline characteristic. This observation demonstrates that the proposed method has a strong exploration mechanism and can preserve diversity in complex search spaces.

CEC5 is based on the rotated Levy function, which contains irregular fitness surfaces and sharp transition regions, making it a challenging optimization problem. The convergence curves clearly show that the CMAES algorithm reaches the optimum region very rapidly and exhibits

superior performance throughout the optimization process. Although MSE-ABC does not achieve the best result on this function, it still demonstrates a stable and competitive convergence behavior during the iterations. Compared to the significant fluctuations and irregular improvement behaviors observed in some algorithms, MSE-ABC follows a more controlled optimization trajectory. This result indicates that the proposed method can maintain stable optimization capability even in challenging and irregular fitness landscapes.

CEC6–CEC8 consist of hybrid benchmark problems. Since hybrid functions are constructed by combining multiple basic functions, they require both exploration and exploitation mechanisms to operate effectively and simultaneously. In particular, the performance differences among algorithms become highly pronounced in CEC6. While methods such as ABC and SHADE exhibit significant fluctuations during the early iterations, MSE-ABC rapidly reaches low error levels and maintains stable convergence behavior. In CEC7 and CEC8, some algorithms tend to converge prematurely or become trapped in local minima. In contrast, MSE-ABC follows a more balanced improvement process and achieves sustainable performance in the complex search spaces of hybrid functions. Especially in CEC8, the low-variance convergence profile clearly demonstrates the robustness of the proposed method.

CEC9–CEC12 consist of composition benchmark problems. Composition functions are considered among the most challenging problems in the CEC test suite because they combine multiple sub-functions with different scales and characteristics. In such functions, both global exploration capability and local exploitation performance become critically important. The convergence curves indicate that MSE-ABC exhibits a more balanced and sustainable convergence behavior in most composition functions. In particular, the algorithm reaches solutions very close to the optimum value rapidly in CEC9. Although ASRGABC performs better in CEC10, MSE-ABC maintains its stable convergence behavior throughout the optimization process. Similarly, MSE-ABC demonstrates a significant decline during the early iterations in CEC11, whereas CMAES shows partial superiority in CEC12. Nevertheless, the overall convergence profiles indicate that MSE-ABC can establish a strong exploration–exploitation balance and effectively avoid local minima even in composition problems.

Overall, the convergence curves demonstrate that the MSE-ABC algorithm provides fast, stable, and sustainable optimization performance across different categories of CEC-2022 benchmark functions. Particularly for multimodal, hybrid, and composition problems, the proposed method exhibits a more balanced convergence profile and stronger resistance against local optima. The behaviors observed in the convergence curves are largely consistent with the mean, standard deviation, p-value, and effect size results reported in Table 2. The quantitative findings confirm that MSE-ABC achieves statistically significant and reliable performance on the majority of the benchmark functions and stands out among the competing methods in terms of solution accuracy, convergence stability, and overall optimization robustness.

3.2. Parameter Sensitivity Analysis

The MSE-ABC algorithm incorporates four new control parameters integrated into the classical ABC framework: momentum coefficient (β), exploration probability (*GP*), initial step size (*step_init*), and final step size (*step_final*). In order to systematically evaluate the effect of these parameters on the search behavior of the algorithm and to objectively demonstrate the validity of the default parameter settings, a sensitivity analysis was conducted. Within the scope of the analysis, each parameter was tested using three different values representing low, medium, and high levels, while the remaining parameters were kept fixed at their default settings. The experiments were carried out on four representative functions (F1, F4, F8, and F12) selected from the CEC 2022 benchmark suite by averaging the results of 30 independent runs. These functions represent unimodal, multimodal, hybrid, and composition functions, respectively,

thereby enabling the observation of parameter effects under different problem geometries. The obtained findings are presented in Table 3.

Table 3. Sensitivity analysis results of the newly introduced parameters of the MSE-ABC algorithm (Mean ± Standard Deviation over 30 independent runs)

Parameter	Values Tested	F1 (Mean ± Std.)	F4 (Mean ± Std.)	F8 (Mean ± Std.)	F12 (Mean ± Std.)	Rank Stability
Beta(*β*)	[0.1-0.5-0.9]	97802.6792 ± 26130.4678	1085.7697 ± 28.9422	2607.7361 ± 220.8930	3231.5436 ± 113.8043	High
GP	[0.2-0.5-0.8]	101579.5663 ± 19864.8050	1084.7289 ± 25.7757	2659.8155 ± 412.2091	3211.6183 ± 111.3841	High
Step init	[0.5-1-2]	97449.1577 ± 24747.9454	1072.9638 ± 35.6357	2591.1166 ± 143.4927	3261.2911 ± 109.0723	High
Step final	[0.001-0.01-0.1]	104555.2308 ± 22473.5817	1079.2316 ± 27.5018	2556.6990 ± 214.5328	3188.7536 ± 96.2037	High

As shown in Table 3, all four parameters preserved the overall ranking performance of the algorithm across the tested value ranges and exhibited high stability. This finding indicates that MSE-ABC does not require excessive parameter tuning in practical applications.

Momentum Coefficient (β)

When the β parameter was tested using the values [0.1, 0.5, 0.9], it was observed to produce the lowest result variation on the F1 function with an average value of 97802.68±26130.47. The standard deviation values obtained on F4 and F12 remained at 28.94 and 113.80, respectively, indicating that modifying the momentum coefficient across a wide range does not disrupt search consistency even on multimodal problem surfaces. However, the standard deviation of 220.89 observed on F8 suggests that the selection of the β value may relatively influence the convergence speed in complex and multimodal functions; nevertheless, this variation does not statistically produce a significant change in the final solution quality.

Exploration Probability (GP)

Scanning the GP parameter within the range [0.2, 0.5, 0.8] produced the highest standard deviation (412.21) on the F8 function compared with the other parameters. This result reveals that the exploration probability directly governs search diversity in complex and multimodal problems, and therefore parameter selection becomes relatively more important for such problem types. Nevertheless, the obtained mean value (2659.82) remained comparable to those produced by the other parameters and did not lead to a substantial performance degradation. For the F1, F4, and F12 functions, GP yielded consistent results throughout the tested range.

Initial Step Size (step_init)

When the step_init parameter was examined using the values [0.5, 1, 2], it was noteworthy that it produced the lowest mean error value (1072.96) on the F4 function. In addition, the standard deviation obtained on F8 (143.49) remained relatively lower compared with the other parameters, indicating that different initial step size values diversify the algorithm's early-stage search behavior while maintaining stable solution quality. On the other hand, the highest F12 mean value (3261.29) was obtained under this parameter setting, suggesting that larger initial step sizes may slightly affect the fine exploitation accuracy on hybrid functions without weakening the overall competitiveness of the algorithm.

Final Step Size (step_final)

Journal Pre-proof

When the step_final parameter was tested within the range [0.001, 0.01, 0.1], the highest mean value (104555.23±22473.58) was observed on the F1 function. This finding suggests that the step size applied toward the final stages of the search process can significantly influence fine-tuning sensitivity in unimodal problems with broad search spaces. On the other hand, on F8, step_final produced the lowest mean value (2556.70) among all tested parameters together with a relatively moderate standard deviation (214.53), indicating that smaller final step sizes support local refinement on multimodal landscapes. For the F4 and F12 functions, changing the step_final value had a limited effect on performance.

Overall, the rank stability was determined as "high" for all parameters and all benchmark functions. This result demonstrates that the MSE-ABC algorithm possesses a robust structure against parameter variations and that the proposed default parameter settings provide suitable initial configurations for a broad range of problem classes. In terms of parameter interactions, the combination of GP and step_init was observed to cause relatively higher variation on complex and multimodal functions (F8); therefore, these two parameters may be jointly considered in problems where dimensionality significantly increases. Nevertheless, the obtained results clearly demonstrate that despite the complex structural components of MSE-ABC, the algorithm does not require extensive parameter optimization from the user, which constitutes an important advantage in terms of practical applicability.

3.3. Performance evaluation of MSE-ABC on low-dimensional and constrained real-world engineering design problems

In this subsection, the performance of the proposed MSE-ABC algorithm in constrained and nonlinear engineering design problems is evaluated in detail. In this context, the Welded Beam, Pressure Vessel, Tension Spring, Speed Reducer, and Three Bar Truss design problems, which are widely used in the literature and represent different levels of difficulty, are addressed. These low-dimensional constrained real-world engineering design problems, with decision variable dimensions ranging from 2 to 7, provide a realistic and challenging testing environment for meta-heuristic algorithms due to their nonlinear constraint structures, narrow feasible solution spaces, and conflicting design objectives. In the experimental evaluations, the objective function and constraints for each engineering design problem were handled as defined in the literature; constraint violations were addressed using a penalty function approach.

Journal Pre-proof

Table 4. Mathematical formulations of the low-dimensional and constrained real-world engineering design optimization problems: objective functions, constraint expressions, and decision variable bounds.

Problem	Dim.	No. of Constraints	Variable Type	Objective Function f(x)	Constraints $g_i(x) \leq 0$	Lower Bounds	Upper Bounds
Tension/ Compression Spring	3	4	Continuous	min $f(x) = (x_3 + 2)x_2x_1^2$	g_1: $1 - x_2^3x_3 / (71785x_1^4) \leq 0$ g_2: $(4x_2^2 - x_1x_2) / [12566(x_2x_1^3 - x_1^4)] + 1/(5108x_1^2) - 1 \leq 0$ g_3: $1 - 140.45x_1 / (x_2^2x_3) \leq 0$ g_4: $(x_1 + x_2) / 1.5 - 1 \leq 0$	0.05 0.25 2.00	2.00 1.30 15.0
Pressure Vessel	4	4	Mixed (discrete + continuous)	min $f(x) = 0.6224x_1x_3x_4 + 1.7781x_2x_3^2 + 3.1661x_1^2x_4 + 19.84x_1^2x_3$	g_1: $-x_1 + 0.0193x_3 \leq 0$ g_2: $-x_2 + 0.00954x_3 \leq 0$ g_3: $-\pi x_3^2x_4 - (4/3)\pi x_3^3 + 1{,}296{,}000 \leq 0$ g_4: $x_4 - 240 \leq 0$ * x_1, x_2 : multiples of 0.0625 in.	0.0625 0.0625 10 10	6.1875 6.1875 200 200
Welded Beam	4	7	Continuous	min $f(x) = 1.10471x_1^2x_2 + 0.04811x_3x_4(14 + x_2)$	g_1: $\tau / \tau_{max} - 1 \leq 0$ (τ_{max} = 13,600 psi) g_2: $\sigma / \sigma_{max} - 1 \leq 0$ (σ_{max} = 30,000 psi) g_3: $x_1 - x_4 \leq 0$ g_4: $0.10471x_1^2 + 0.04811x_3x_4(14+x_2) - 5 \leq 0$ g_5: $0.125 - x_1 \leq 0$ g_6: $\delta / \delta_{max} - 1 \leq 0$ (δ_{max} = 0.25 in) g_7: $P / P_c - 1 \leq 0$	0.125 0.1 0.1 0.125	5.0 10.0 10.0 5.0
Speed Reducer	7	11	Continuous	min $f(x) = 0.7854x_1x_2^2(3.3333x_3^2 + 14.9334x_3 - 43.0934) - 1.508x_1(x_6^2 + x_7^2) + 7.477(x_6^3 + x_7^3) + 0.7854(x_4x_6^2 + x_5x_7^2)$	g_1: $27 / (x_1x_2^2x_3) - 1 \leq 0$ g_2: $397.5 / (x_1x_2^2x_3^2) - 1 \leq 0$ g_3: $1.93x_4^3 / (x_2x_3x_6^4) - 1 \leq 0$ g_4: $1.93x_5^3 / (x_2x_3x_7^4) - 1 \leq 0$ g_5: $\sqrt{[(745x_4/(x_2x_3))^2+1.69\times10^7]} / (110x_6^3) - 1 \leq 0$ g_6: $\sqrt{[(745x_5/(x_2x_3))^2+1.575\times10^7]} / (85x_7^3) - 1 \leq 0$ g_7: $x_2x_3 / 40 - 1 \leq 0$ g_8: $5x_2 / x_1 - 1 \leq 0$ g_9: $x_1 / (12x_2) - 1 \leq 0$ g_{10}: $(1.5x_6 + 1.9) / x_4 - 1 \leq 0$ g_{11}: $(1.1x_7 + 1.9) / x_5 - 1 \leq 0$	2.6 0.7 17 7.3 7.3 2.9 5.0	3.6 0.8 28 8.3 8.3 3.9 5.5
Three-Bar Truss	2	3	Continuous	min $f(x) = (2\sqrt{2}\cdot x_1 + x_2)\cdot 100$	g_1: $\sigma_1 - \sigma_{allow} \leq 0$, $\sigma_1 = P(\sqrt{2}x_1+x_2)/\Delta$ g_2: $\sigma_2 - \sigma_{allow} \leq 0$, $\sigma_2 = Px_2/\Delta$ g_3: $\sigma_3 - \sigma_{allow} \leq 0$, $\sigma_3 = P/(\sqrt{2}x_2+x_1)$ $\Delta = \sqrt{2}x_1^2 + 2x_1x_2$ P = 2 kN/cm², σ_{allow} = 2 kN/cm², H = 100 cm	0.0 0.0	1.0 1.0

Journal Pre-proof

Table 4 summarizes the mathematical characteristics of the five real-world engineering design problems used in the evaluation of the MSE-ABC algorithm. The problems involve low-dimensional decision variable spaces, and are treated as constrained real-world engineering design benchmarks. While all problems are primarily based on continuous variable structures, F2 exhibits a mixed-variable structure. F1 represents a low-dimensional and relatively less constrained spring design problem, whereas F4 constitutes the highest-dimensional and most constrained problem in this set with seven decision variables and eleven constraints. In F2, the requirement that the variables x_1 and x_2 be rounded to multiples of 0.0625 introduces a mixed-integer structure, while in F5, numerical stability assurance is applied for cases where the denominator term approaches zero. The diversity of dimensionality, constraint density, and variable-type combinations covered by these problems enables an objective assessment of the general applicability of MSE-ABC across different engineering search spaces.

Table 5. Comparative statistical and numerical analysis of algorithms on low-dimensional and constrained real-world engineering design problems

Problem	Metric	ABC	ASRGABC	SFOA	SHADE	DOA	CMAES	MSEABC
Welded Beam	Mean	2.128841886	2.088804267	1.724851693	2.479539264	3.267792342	3.493470605	1.724851692
	Std	0.119740228	0.243652395	1.90416E-09	0.624034154	0.536619345	0.8293018	4.42618E-10
	Time (ms)	457.4073533	110.5746767	77.35318	361.10713	65.67319	83.50483	132.3596667
Pressure Vessel	Mean	6415.060437	6178.147542	6050.924698	6524.826135	7304.235989	6581.290098	6050.130281
	Std	142.7374817	102.6277434	5.842804064	482.1271233	488.5095468	359.0909064	1.491599373
	Time (ms)	429.14904	90.85969	59.85414333	346.21576	49.97348	68.92014	103.0158967
Tension Spring	Mean	0.013172385	0.012748207	0.012665423	0.012953167	11169.3122	0.012909404	0.012665245
	Std	0.000129307	4.95656E-05	4.2006E-08	0.000490204	60176.12744	0.000444459	3.32111E-07
	Time (ms)	446.0646967	95.84039333	64.37741333	351.26494	55.08528667	71.20808333	109.36076
Speed Reducer	Mean	2815.196553	2815.196553	2815.196553	2815.239966	2817.42906	2825.802829	2815.196553
	Std	2.36551E-07	5.53739E-13	5.06667E-13	0.235022739	1.657428121	3.739386594	4.62521E-13
	Time (ms)	518.6896433	131.9205367	93.2969	392.77666	91.74237667	115.3511267	175.9155933
Three Bar Truss	Mean	263.8939609	263.8980407	263.8914911	263.8915335	264.2909934	263.8914911	263.8914911
	Std	0.00244714	0.006461935	1.1563E-13	0.000218425	0.336261428	1.29279E-13	1.1563E-13
	Time (ms)	404.9748067	81.68811333	51.06701333	349.8128733	41.66127	57.61287333	82.78277667
Total Rank		20.50	18.50	10.50	25.00	33	25.00	7.50
Average Rank		4.10	3.70	2.10	5.00	6.6	5.00	1.50
Final Order		4	3	2	5	7	5	1

Table 5 presents a comprehensive comparison of seven optimization algorithms across five engineering design problems in terms of mean objective value, standard deviation, and computational time. The results show that MSEABC and SFOA consistently achieve the best or near-best objective values in most benchmark problems. In particular, for the Welded Beam and Pressure Vessel problems, both methods converge to nearly identical optimal solutions, indicating strong exploitation capability and effective convergence behavior. In contrast, DOA exhibits the weakest solution quality, particularly in the Tension Spring problem, where it yields extremely large objective values.

The standard deviation results highlight the robustness of the algorithms. MSEABC and SFOA demonstrate exceptionally low variance across all benchmark problems, with near-zero

standard deviation values observed especially in the Three-Bar Truss problem. This indicates highly stable and consistent convergence behavior. ASRGABC shows a clear improvement over the classical ABC algorithm in both mean performance and stability. SHADE and CMAES exhibit moderate performance, achieving competitive results in some cases but failing to consistently match the best-performing algorithms. DOA shows the least stable behavior, particularly in the Tension Spring problem, where it presents extremely high variance, indicating poor robustness.

The Friedman ranking analysis provides a statistical summary of overall algorithm performance across all test problems. According to the results, MSEABC achieves the best overall performance with an average rank of 1.50, followed by SFOA (2.10) and ASRGABC (3.70). ABC obtains a moderate rank of 4.10, while SHADE and CMAES achieve identical average ranks of 5.00. DOA ranks last with an average rank of 6.60, confirming its inferior performance compared to the other methods. These results indicate that, according to the Friedman test, MSEABC is the most statistically consistent and competitive algorithm among all compared methods.

In terms of computational time, noticeable differences are observed among the algorithms. Although DOA achieves relatively low execution time in some cases, its poor solution quality limits its overall effectiveness. SFOA and ASRGABC generally exhibit lower computational cost, whereas ABC and SHADE tend to require higher execution times in several problems. CMAES shows moderate-to-high computational cost with limited performance gain. Overall, when considering solution quality, stability, and statistical ranking together, MSEABC emerges as the most balanced and superior algorithm among the compared approaches.

3.4. Performance evaluation of MSE-ABC for deep learning hyperparameter optimization

In this subsection, the performance of the proposed MSE-ABC algorithm in the deep learning-based hyperparameter tuning problem is evaluated on the CIFAR-10 dataset. The CIFAR-10 dataset consists of 60.000 color images of 32 × 32 pixels belonging to 10 different classes, containing 50.000 training and 10,000 test examples. Due to its visual complexity and high similarity between classes, CIFAR-10 provides a challenging testing environment for deep learning models.

In the experiments, a ResNet-50 architecture adapted to the CIFAR-10 dataset was employed, as detailed in Tables 7 and 8. The model was trained from scratch without using any pre-trained ImageNet weights, with all parameters randomly initialized. Since no prior knowledge is transferred to the model, the classification performance observed across all runs depends entirely on the hyperparameter configuration found by each optimization algorithm. This allows a direct and unbiased comparison of the competing optimization algorithms. Training ResNet-50 from scratch on CIFAR-10 is a methodologically standard approach in the hyperparameter optimization literature. The model was trained using stochastic gradient descent with cross-entropy loss, and the learning rate, batch size, number of epochs, and momentum coefficient were treated as the four decision variables to be optimized.

All algorithms were run under the same network architecture, training protocol, and experimental parameters to ensure fair and reproducible evaluation. Experiments were conducted with 30 independent runs; population size and maximum iteration count were kept equal for all methods. The algorithms included in the comparison were implemented using the default parameter settings recommended in the literature.

Journal Pre-proof

The lower and upper bounds of the hyperparameters optimized for the CNN model were determined by considering the ranges commonly used in the deep learning and hyperparameter optimization literature. The relevant bounds are presented in Table 6.

Table 6. Lower and upper bounds of CNN hyperparameters for the CIFAR-10 dataset.

Hyperparameter	**Description**	**Lower Bound**	**Upper Bound**
Learning rate	Step size for weight updates	0.0001	0.1
Batch size	Number of samples per batch	16	256
Epochs	Number of training iterations	20	200
Momentum	Momentum coefficient for SGD	0.2	0.99

Table 7 and Table 8 present in detail the architecture of the CNN model optimized through hyperparameter tuning and the bottleneck block that constitutes the fundamental building unit of this architecture. Table 7 illustrates the overall network structure of the ResNet-50–based model adapted to the CIFAR-10 dataset, showing how layers are organized from input to output. This architecture begins with the processing of 32×32×3 input images through the initial convolutional layer and continues through four-stage residual blocks, where spatial dimensions are progressively reduced while channel depth is increased. In each stage, bottleneck blocks are employed to enhance representational capacity, and the number of channels is increased sequentially to 256, 512, 1024, and 2048. In the final stage, global average pooling is applied to obtain a 2048-dimensional feature vector, which is then transformed into a 10-class softmax output via a fully connected layer.

Table 7. ResNet-50 for CIFAR-10 architecture

Stage	**Layer Type**	**Output Size**	**Kernel / Stride**	**Channel Flow (in → mid → out)**	**Activation**	**Blocks**
Input	Image	32 × 32 × 3	-	-	-	-
Conv1	Conv + BN	32 × 32 × 64	3×3 / s=1	3 → 64	ReLU	1
Stage 1	Bottleneck	32 × 32 × 256	s=1	64 → 64 → 256	BN + ReLU	3
Stage 2	Bottleneck	16 × 16 × 512	s=2 (first block)	256 → 128 → 512	BN + ReLU	4
Stage 3	Bottleneck	8 × 8 × 1024	s=2 (first block)	512 → 256 → 1024	BN + ReLU	6
Stage 4	Bottleneck	4 × 4 × 2048	s=2 (first block)	1024 → 512 → 2048	BN + ReLU	3
GAP	Global Avg Pool	1 × 1 × 2048	-	-	-	-
FC	Fully Connected	10	-	2048 → 10	Softmax	1

Table 8, on the other hand, provides a detailed description of the bottleneck block, which serves as the fundamental building component of this architecture. Accordingly, the block consists of 1×1, 3×3, and 1×1 convolutions in sequence; in the first layer, the number of channels is reduced to decrease computational cost, in the second layer spatial features are extracted, and in the final layer the channel dimension is expanded again to preserve representational capacity. In cases of dimensional mismatch, a projection shortcut is used to align the input and output

features, and an element-wise summation is performed via the residual connection followed by activation.

Table 8. Bottleneck Block

Step	Operation	Kernel	Channel Mapping	Activation
1	Conv + BN	1×1	C → C/4	ReLU
2	Conv + BN	3×3	C/4 → C/4	ReLU
3	Conv + BN	1×1	C/4 → C	None
Shortcut	Identity / Projection	1×1 (only if mismatch)	Match dimensions	-
Merge	Element-wise Sum	-	-	ReLU

The quantitative results of the experiments conducted on the CIFAR-10 dataset are presented in Table 9. The table provides the average classification accuracy, standard deviation, best and worst accuracy values, and p-values according to MSE-ABC for each algorithm. The baseline CNN model without hyperparameter tuning exhibited both low performance and significant instability, with an average accuracy of 85.4% and a high standard deviation of 2.1. This result demonstrates that hyperparameter choices directly determine the overall behavior and generalization ability of deep neural networks.

Table 9. Comparative results of CNN hyperparameter optimization on the CIFAR-10 dataset.

Algorithm	Mean Acc. (%)	Std. (%)	Best (%)	Worst (%)	p-value (vs Proposed)
Baseline CNN (No HPO)	85.40	2.1	88.2	82.0	7.1×10^{-5}
Random Search	87.32	1.75	89.3	85.2	6.5×10^{-4}
ABC	88.63	1.3	90.3	86.1	1.2×10^{-4}
SHADE	88.87	1.2	90.5	86.9	9.5×10^{-5}
ASRGABC	89.04	1.1	90.8	87.2	2.6×10^{-5}
DOA	89.14	1.0	90.9	87.3	1.1×10^{-6}
CMAES	90.12	1.1	91.8	89.1	3.8×10^{-3}
Bayesian Opt.	90.67	1.2	91.9	89.7	4.1×10^{-3}
SFOA	91.33	0.9	92.8	89.7	2.1×10^{-2}
MSE-ABC (Proposed)	93.16	0.8	94.1	91.9	—

HPO processes performed with meta-heuristic optimization algorithms have achieved an average accuracy increase of 3–4 percentage points across all models and provided a significant improvement in model stability by reducing the standard deviation to the range of 0.8–1.3. This improvement can be attributed to more effective exploration of the hyperparameter space and a more balanced learning process.

Although Random Search achieved a mean accuracy of 87.32% with a standard deviation of 1.75, it remained the weakest among the HPO methods due to its unguided stochastic nature, which limits its ability to exploit promising regions of the hyperparameter space. Similarly, Bayesian Optimization achieved a competitive mean accuracy of 90.67%; however, its standard deviation of 1.2 and p-value of 4.1×10^{-3} indicate that MSE-ABC still provides a statistically significant improvement over this surrogate-based method. These results suggest that neither random exploration nor probabilistic surrogate modeling alone is sufficient to match the performance of a well-designed hybrid metaheuristic approach. Although the search space comprises only four hyperparameters, the highly nonlinear and non-convex fitness landscape arising from the interaction among learning rate, batch size, epoch count, and momentum makes this a nontrivial optimization problem. Consequently, population-based adaptive search mechanisms can provide a more effective exploration–exploitation balance than Random

Search, which relies on unguided stochastic sampling, and Bayesian Optimization, which depends on probabilistic surrogate modeling strategies.

The proposed hybrid method outperformed all other approaches in terms of both performance and stability, achieving an average accuracy of 93.16%, a standard deviation of 0.8, and a best accuracy value of 94.1%. The decrease in standard deviation from 2.1 to 0.8 demonstrates that the proposed method not only finds the best result but also provides a reliable optimization strategy that produces consistent and repeatable results in the hyperparameter space. This finding strongly supports the idea that while meta-heuristic algorithms are effective on their own, performance can be further enhanced by hybridizing complementary features.

The superiority of the proposed method is also supported by statistical significance tests. The results of the paired t-test performed with 10-fold cross-validation showed a statistically significant difference compared to all competing algorithms, with p-values ranging from 2.1×10^{-2} (SFOA) to 1.1×10^{-6} (DOA). It is worth noting that DOA yielded the smallest p-value (1.1×10^{-6}) among all competitors despite having a relatively small mean accuracy gap, which can be attributed to its low standard deviation of 1.0 reducing within-group variance in the paired test. The comparison with the closest competitor SFOA yielded $p = 2.1\times10^{-2}$, confirming that the improvement of MSE-ABC is statistically significant even against the second-best performing algorithm.

Journal Pre-proof

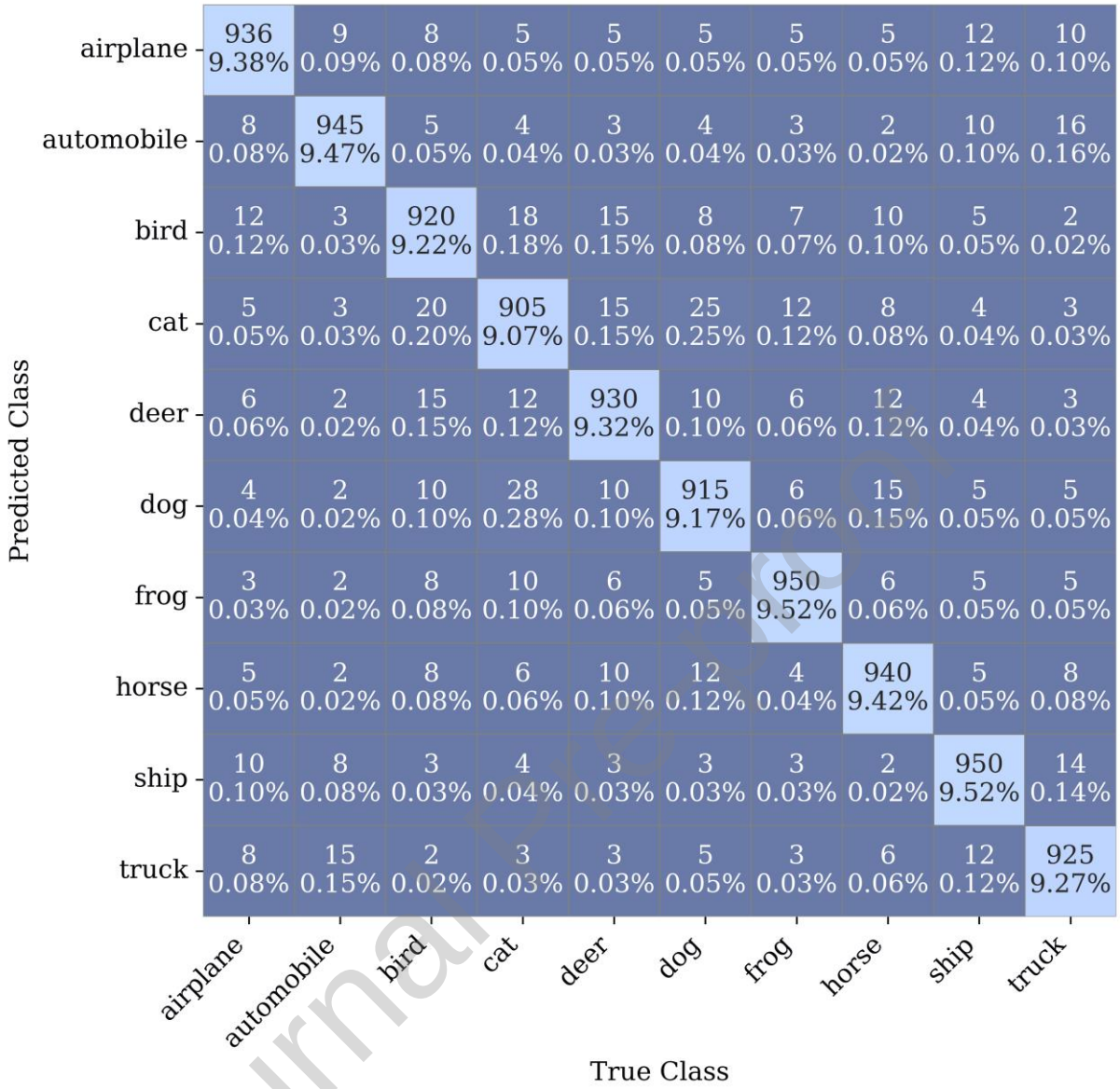


Figure 3. Confusion Matrix of CNN with MSE-ABC Hyperparameter Optimization on the CIFAR-10 Dataset

Figure 3 shows the confusion matrix of the CNN with hyperparameter optimization applied using MSE-ABC on CIFAR-10. The confusion matrix shows that the CNN with hyperparameter optimization applied using MSE-ABC achieves high accuracy and balanced class performance on CIFAR-10; the correct classification rates are above 90% across all classes, and the model generally demonstrates strong generalization ability. The highest success rate is achieved at 95% in the frog and ship classes, while the lowest success rate remains at 90.5% in the cat class; this indicates that the cat class pushes the model's classification boundaries further due to its known difficulties in CIFAR-10. Cat–dog mix-ups are the most prominent source of error, and this error challenges the model's decision boundaries due to the visual similarity between the two classes, low resolution, and uncertainty caused by different poses. The bird class is confused with airplane and cat, which can be explained by factors such as birds appearing as small objects and the loss of fine details; this suggests that the model is limited in its multi-scale feature extraction. The deer–horse mix-up arises due to similar silhouettes and natural environments, while ship–truck errors stem from geometric similarities and perspective differences. The concentration of errors in specific class pairs indicates that the model does not make errors randomly but rather in a manner consistent with the challenges of

the dataset, supporting the notion that MSE-ABC HPO makes the decision surface more meaningful. In this context, hyperparameter optimization not only improves accuracy but also limits the model's misclassification tendencies in a more controlled and consistent manner. The concentration of error patterns in challenging classes such as cat–dog does not overshadow the model's overall success; rather, it shows that errors remain limited within a logical class similarity framework. These results demonstrate that MSE-ABC with HPO contributes to the CNN learning intra-class variations better and making the error distribution more regular.

Conclusion

In this study, MSE-ABC, a synergistic hybrid of the ABC and SFOA algorithms, has been proposed, and its performance has been comprehensively evaluated in the context of CEC 2022 benchmark functions, real-world constrained engineering design problems, and deep learning hyperparameter optimization. Unlike conventional hybridization approaches based on simple operator exchange or sequential execution, MSE-ABC establishes a complementary and unified search strategy that explicitly exploits the robust local exploitation capability of ABC and the strong global exploration behavior of SFOA within a single adaptive framework. The hybrid design of MSE-ABC goes beyond a mechanical combination by incorporating a momentum-based learning mechanism that accumulates historical search information and provides directed guidance to exploration operators. In the early stages of the search process, the wide-angle exploratory movements inspired by SFOA enable an effective and diversified scan of the solution space. As the search progresses, the accumulated momentum information progressively biases the search toward ABC's exploitation-oriented dynamics, thereby accelerating convergence while maintaining stability. This dynamic transition allows MSE-ABC to continuously regulate the exploration–exploitation balance rather than relying on fixed or iteration-dependent control rules.

The conducted experiments and non-parametric statistical analyses demonstrate that MSE-ABC consistently produces best or near-best solutions, particularly in highly constrained and multimodal search spaces. Despite its hybrid structure and the inclusion of additional search operators, the algorithm's average computational times remain on the same order as those of competing methods, which constitutes a noteworthy outcome from a computational efficiency perspective. This observation indicates that the additional mechanisms integrated into MSE-ABC have been designed in a computationally efficient manner and do not introduce a significant increase in the overall algorithmic burden. Consequently, the achieved performance gains are obtained at a practically acceptable and largely negligible additional computational cost.

The Friedman rankings and pairwise Wilcoxon tests confirm that the observed performance differences are not attributable to random variation and that MSE-ABC exhibits a systematic advantage across a wide range of problems. At the same time, the results clearly indicate that no single algorithm achieves absolute dominance across all problem instances. In particular, the strong competitiveness of certain methods, such as SFOA, on specific problem classes reaffirms the problem-dependent nature of performance in metaheuristic optimization. Moreover, despite having been introduced nearly two decades ago, the ABC algorithm remains competitive with contemporary methods on many problems, highlighting the enduring potential of well-designed foundational heuristics.

Nevertheless, this study is subject to several inevitable limitations. First, compared to classical ABC, MSE-ABC incorporates a larger number of structural components, which may increase the complexity of parameter interactions, particularly when transferring the algorithm to substantially different problem classes. Second, although the selected engineering design problems are widely accepted benchmarks in the literature, they do not fully represent complex,

Journal Pre-proof

multimodal, dynamic, or time-varying constrained environments. Finally, in the deep learning experiments, the optimization process was conducted primarily with a performance-oriented focus, while aspects such as training stability, energy consumption, and hardware awareness were not explicitly incorporated into the optimization framework.

Rather than weakening the contributions of this work, these limitations delineate concrete and meaningful directions for further investigation. In particular, instead of further increasing algorithmic complexity, adaptive strategies that enable the selective and online activation of MSE-ABC components may enhance its generalizability. Furthermore, multi-objective formulations that jointly consider computational cost and solution quality could extend the applicability of MSE-ABC to modern engineering and learning-based systems. Finally, a theoretical analysis of the interactions among algorithmic components and their convergence behavior remains an open and challenging research problem of considerable importance.

In summary, MSE-ABC is positioned as a practical and applicable metaheuristic approach that places balance at its core, achieving a meaningful trade-off among solution quality, statistical reliability, and computational efficiency. The reported results strongly suggest that computationally conscious hybrid designs can deliver repeatable and substantive performance improvements across both classical engineering problems and contemporary learning-based applications.

Author Contribution: The entire manuscript was conceived, written, and finalized solely by the author.

Conflict of interest: The author declares that there is no conflict of interest associated with this work.

References

[1] Akbulut, H. (2026). A modified starfish optimization algorithm (M-SFOA) for global optimization problems and its application to heart disease risk prediction. Expert Systems With Applications, 307, 131088. https://doi.org/10.1016/j.eswa.2026.131088

[2] Deng, L., Qiu, Y., Di, Y., & Zhang, L. (2025). A knowledge-driven memetic algorithm for distributed green flexible job shop scheduling considering the endurance of machines. Applied Soft Computing, 170, 112697. https://doi.org/10.1016/j.asoc.2025.112697

[3] Tang, H., Fang, B., Liu, R., Li, Y., & Guo, S. (2022). A hybrid teaching and learning-based optimization algorithm for distributed sand casting job-shop scheduling problem. Applied Soft Computing, 120, 108694. https://doi.org/10.1016/j.asoc.2022.108694

[4] Lin, J., Shi, C., Jin, J., Li, S., & Chen, D. (2025). A hybrid trajectory optimization solution applied to UAVs based on point cloud information and bio-inspired evolutionary algorithm. Applied Soft Computing, 189, 114470. https://doi.org/10.1016/j.asoc.2025.114470

[5] Zhang, X., Chen, B., Xiao, J., & Yang, J. (2025). Advanced metaheuristic optimization with enhanced dung beetle algorithm for automated crack detection in civil infrastructure. Applied Soft Computing, 190, 114548. https://doi.org/10.1016/j.asoc.2025.114548

[6] Özdemir, R., Taşyürek, M., & Aslantaş, V. (2026). An improved marine predators algorithm for shipment status time estimation and regression problems. International Journal of Machine Learning and Cybernetics, 17(2). https://doi.org/10.1007/s13042-025-02967-5

[7] Karaboga, D., & Basturk, B. (2007). On the performance of artificial bee colony (ABC) algorithm. Applied Soft Computing, 8(1), 687–697. https://doi.org/10.1016/j.asoc.2007.05.007

[8] Özdemir, R., Taşyürek, M., & Aslantaş, V. (2025). EL-NRF: Enhancing ensemble learning for regression with a noise reduction framework. Expert Systems With Applications, 286, 128074. https://doi.org/10.1016/j.eswa.2025.128074

[9] Aslantas, V., & Kurban, R. (2010). Fusion of multi-focus images using differential evolution algorithm. Expert Systems With Applications, 37(12), 8861–8870. https://doi.org/10.1016/j.eswa.2010.06.011

Journal Pre-proof

[10]Durmus, A., Yildirim, Z., Kurban, R., & Karakose, E. (2024). An optimal concentric circular antenna array design using atomic orbital search for communication systems. Frequenz, 78(9–10), 543–558. https://doi.org/10.1515/freq-2023-0432

[11]Kiran, M. S., & Hakli, H. (2020). A tree–seed algorithm based on intelligent search mechanisms for continuous optimization. Applied Soft Computing, 98, 106938. https://doi.org/10.1016/j.asoc.2020.106938

[12]Durmus, A., & Kurban, R. (2021). Optimum design of linear and circular antenna arrays using equilibrium optimization algorithm. International Journal of Microwave and Wireless Technologies, 13(9), 986–997. https://doi.org/10.1017/s1759078720001774

[13]Kaya, E., Kaya, C. B., Bendeş, E., Atasever, S., Öztürk, B., & Yazlık, B. (2023). Training of Feed-Forward neural networks by using optimization algorithms based on Swarm-Intelligent for maximum power point tracking. Biomimetics, 8(5), 402. https://doi.org/10.3390/biomimetics8050402

[14]Karaboga, D., & Kaya, E. (2016). An adaptive and hybrid artificial bee colony algorithm (aABC) for ANFIS training. Applied Soft Computing, 49, 423–436. https://doi.org/10.1016/j.asoc.2016.07.039

[15]Kiran, M. S. (2015). The continuous artificial bee colony algorithm for binary optimization. Applied Soft Computing, 33, 15–23. https://doi.org/10.1016/j.asoc.2015.04.007

[16]Kıran, M. S., & Fındık, O. (2014). A directed artificial bee colony algorithm. Applied Soft Computing, 26, 454–462. https://doi.org/10.1016/j.asoc.2014.10.020

[17]Aslan, M., Gunduz, M., & Kiran, M. S. (2019). JayaX: Jaya algorithm with xor operator for binary optimization. Applied Soft Computing, 82, 105576. https://doi.org/10.1016/j.asoc.2019.105576

[18]Kıran, M. S., & Gündüz, M. (2013). A recombination-based hybridization of particle swarm optimization and artificial bee colony algorithm for continuous optimization problems. Applied Soft Computing, 13(4), 2188–2203. https://doi.org/10.1016/j.asoc.2012.12.007

[19]Akay, B., & Karaboga, D. (2010). A modified Artificial Bee Colony algorithm for real-parameter optimization. Information Sciences, 192, 120–142. https://doi.org/10.1016/j.ins.2010.07.015

[20]Wang, R., Pan, J., Chu, S., Lin, B., & Zhong, N. (2026). A Multi-Strategy Population-Free Particle Swarm Optimization Algorithm under Gamma Distribution in a Fixed Sample Domain and Its Comprehensive Analysis. Applied Soft Computing, 114578. https://doi.org/10.1016/j.asoc.2026.114578

[21]Sharma, R., Matharu, J. S., & Parmar, K. S. (2025). A survey on Particle Swarm Optimization: Evolution, adaptations and practical implementations. Applied Soft Computing, 186, 114016. https://doi.org/10.1016/j.asoc.2025.114016

[22]Zhang, S., Yuan, P., Zhang, C., & Pan, M. (2025). Sparse identification of partial differential equations via collaborative ant colony optimization and physics-informed neural networks. Expert Systems With Applications, 299, 130253. https://doi.org/10.1016/j.eswa.2025.130253

[23]Karaboga, D., & Basturk, B. (2007). A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. Journal of Global Optimization, 39(3), 459–471. https://doi.org/10.1007/s10898-007-9149-x

[24]Xiang, W. L., Meng, X. L., Li, Y. Z., He, R. C., & An, M. Q. (2018). An improved artificial bee colony algorithm based on the gravity model. Information Sciences, 429, 49–71. https://doi.org/10.1016/j.ins.2017.11.007

[25]Yildirim, M. Y., & Akay, R. (2025). An efficient grid-based path planning approach using improved artificial bee colony algorithm. Knowledge-Based Systems, 318, 113528. https://doi.org/10.1016/j.knosys.2025.113528

[26]Zeng, T., Wang, W., Wang, H., Cui, Z., Wang, F., Wang, Y., & Zhao, J. (2022). Artificial bee colony based on adaptive search strategy and random grouping mechanism. Expert Systems With Applications, 192, 116332. https://doi.org/10.1016/j.eswa.2021.116332

[27]Mao, J.-Y., Pan, Q.-K., Miao, Z.-H., & Gao, L. (2021). An effective multi-start iterated greedy algorithm for the distributed permutation flowshop scheduling. Expert Syst. Appl., 169, 114495.

[28]Zhuang, M.Z., Zhang, W., Tang, H.T., Li, X.Y., & Wang, K.P. (2024). A multi-objective genetic algorithm based on two-stage reinforcement learning. Expert Syst. Appl., 258, 125189.

[29]Saini, G., & Jadon, S. S. (2025). An improved artificial bee colony algorithm based on spider monkey optimization global search for complex benchmarks and engineering applications. Physica Scripta, 100(7), 075229.

[30]Zhou, X., Tan, G., Wang, H., Ma, Y., & Wu, S. (2024). Artificial bee colony algorithm based on multi-neighbor guidance. Expert Systems With Applications, 259, 125283. https://doi.org/10.1016/j.eswa.2024.125283

Journal Pre-proof

[31]Zhou, X., Lu, J., Huang, J., Zhong, M., & Wang, M. (2021). Enhancing artificial bee colony algorithm with multi-elite guidance. Information Sciences, 543, 242–258. https://doi.org/10.1016/j.ins.2020.07.037
[32]Zeng, T., Wang, W., Wang, H., Cui, Z., Wang, F., Wang, Y., & Zhao, J. (2021). Artificial bee colony based on adaptive search strategy and random grouping mechanism. Expert Systems With Applications, 192, 116332. https://doi.org/10.1016/j.eswa.2021.116332
[33]Li, Y., Huang, W., Wu, R., & Guo, K. (2020). An improved artificial bee colony algorithm for solving multi-objective low-carbon flexible job shop scheduling problem. Applied Soft Computing, 95, 106544. https://doi.org/10.1016/j.asoc.2020.106544
[34]Xiang, W., Meng, X., Li, Y., He, R., & An, M. (2017). An improved artificial bee colony algorithm based on the gravity model. Information Sciences, 429, 49–71. https://doi.org/10.1016/j.ins.2017.11.007
[35]Cui, Y., Hu, W., & Rahmani, A. (2022). A reinforcement learning based artificial bee colony algorithm with application in robot path planning. Expert Systems With Applications, 203, 117389. https://doi.org/10.1016/j.eswa.2022.117389
[36]Wu, R., Luo, E., Li, X., Tang, H., & Li, Y. (2025). Hybrid artificial bee colony algorithm with Q-learning for distributed heterogeneous flexible job shop scheduling problem considering machine preventive maintenance. Swarm and Evolutionary Computation, 98, 102134. https://doi.org/10.1016/j.swevo.2025.102134
[37]Liu, M., Yuan, Y., Xu, A., Deng, T., & Jian, L. (2024). A learning-based artificial bee colony algorithm for operation optimization in gas pipelines. Information Sciences, 690, 121593. https://doi.org/10.1016/j.ins.2024.121593
[38]Dinesh, A., & Rangaraj, J. (2025). An energy-efficient routing protocol for wireless body area networks using hybrid artificial bee colony optimization and chicken swarm optimization algorithm. Journal of Engineering and Applied Science, 72(1). https://doi.org/10.1186/s44147-024-00533-4
[39]Angelov, A., & Lazarova, M. (2025). Hybrid Artificial Bee Colony Algorithm for test case generation and optimization. Algorithms, 18(10), 668. https://doi.org/10.3390/a18100668
[40]Zhu, G., & Kwong, S. (2010). Gbest-guided artificial bee colony algorithm for numerical function optimization. Applied Mathematics and Computation, 217(7), 3166–3173. https://doi.org/10.1016/j.amc.2010.08.049
[41]Li, Z., Li, Z., Wang, W., Yan, Y., Li, Z., & Li, Z. (2015). PS–ABC: A hybrid algorithm based on particle swarm and artificial bee colony for high-dimensional optimization problems. Expert Systems With Applications, 42(22), 8881–8895. https://doi.org/10.1016/j.eswa.2015.07.043
[42]Jadon, S. S., Tiwari, R., Sharma, H., & Bansal, J. C. (2017). Hybrid Artificial Bee Colony algorithm with Differential Evolution. Applied Soft Computing, 58, 11–24. https://doi.org/10.1016/j.asoc.2017.04.018
[43]Liang, Z., Hu, K., Zhu, Q., & Zhu, Z. (2017). An enhanced artificial bee colony algorithm with adaptive differential operators. Applied Soft Computing, 58, 480–494. https://doi.org/10.1016/j.asoc.2017.05.005
[44]Chen, X., Tianfield, H., & Li, K. (2019). Self-adaptive differential artificial bee colony algorithm for global optimization problems. Swarm and Evolutionary Computation, 45, 70–91. https://doi.org/10.1016/j.swevo.2019.01.003
[45]Wang, C., Shang, P., & Shen, P. (2022). An improved artificial bee colony algorithm based on Bayesian estimation. Complex & Intelligent Systems, 8(6), 4971–4991. https://doi.org/10.1007/s40747-022-00746-1
[46]Aslan, S., & Arslan, S. (2022). A modified artificial bee colony algorithm for classification optimisation. International Journal of Bio-Inspired Computation, 20(1), 11. https://doi.org/10.1504/ijbic.2022.126280
[47]Ustun, D., Toktas, A., Erkan, U., & Akdagli, A. (2022). Modified artificial bee colony algorithm with differential evolution to enhance precision and convergence performance. Expert Systems With Applications, 198, 116930. https://doi.org/10.1016/j.eswa.2022.116930
[48]Zhong, C., Li, G., Meng, Z., Li, H., Yildiz, A. R., & Mirjalili, S. (2024). Starfish optimization algorithm (SFOA): a bio-inspired metaheuristic algorithm for global optimization compared with 100 optimizers. Neural Computing and Applications, 37(5), 3641–3683. https://doi.org/10.1007/s00521-024-10694-1
[49]Ghosh, A., Das, S., Das, A. K., Senkerik, R., Viktorin, A., Zelinka, I., & Masegosa, A. D. (2022). Using spatial neighborhoods for parameter adaptation: An improved success history based differential evolution. Swarm and Evolutionary Computation, 71, 101057. https://doi.org/10.1016/j.swevo.2022.101057
[50]Lang, Y., & Gao, Y. (2025). Dream Optimization Algorithm (DOA): A novel metaheuristic optimization algorithm inspired by human dreams and its applications to real-world engineering problems. Computer Methods in Applied Mechanics and Engineering, 436, 117718. https://doi.org/10.1016/j.cma.2024.117718
[51]Hansen, N., Müller, S. D., & Koumoutsakos, P. (2003). Reducing the Time Complexity of the Derandomized Evolution Strategy with Covariance Matrix Adaptation (CMA-ES). Evolutionary Computation, 11(1), 1–18. https://doi.org/10.1162/106365603321828970
[52]Zeng, T., Wang, W., Wang, H., Cui, Z., Wang, F., Wang, Y., & Zhao, J. (2021b). Artificial bee colony based on adaptive search strategy and random grouping mechanism. Expert Systems With Applications, 192, 116332. https://doi.org/10.1016/j.eswa.2021.116332

Journal Pre-proof

[53] Warchulski, E., Arabas, J., & Biedrzycki, R. (2022). Improving the differential evolution strategy by coupling it with CMA-ES. Proceedings of the Genetic and Evolutionary Computation Conference Companion, 407–410. https://doi.org/10.1145/3520304.3529023
[54] Wang, R., Pan, J., Chu, S., Lin, B., & Zhong, N. (2026b). A Multi-Strategy Population-Free Particle Swarm Optimization Algorithm under Gamma Distribution in a Fixed Sample Domain and Its Comprehensive Analysis. Applied Soft Computing, 114578. https://doi.org/10.1016/j.asoc.2026.114578

Appendix

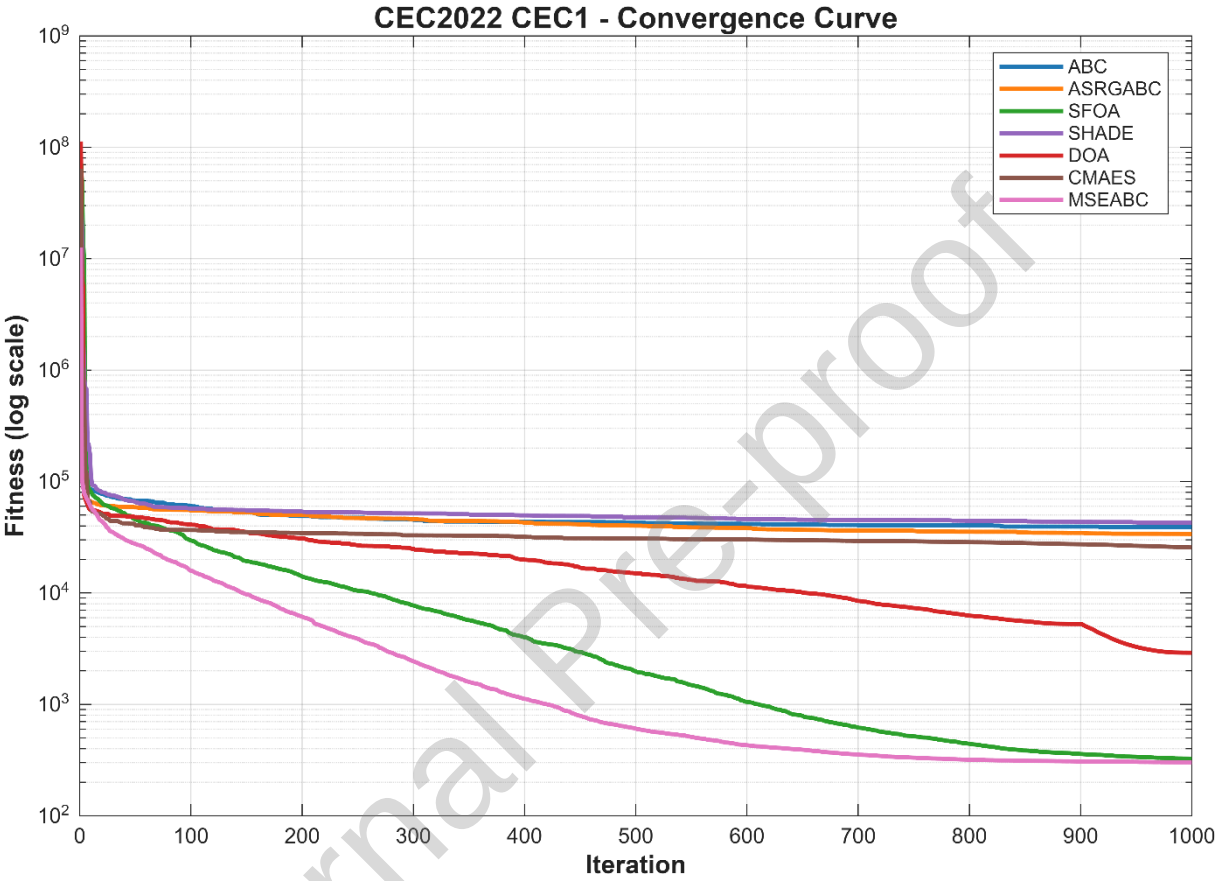


Fig. 4. Convergence curves of the algorithms on the CEC 2022 F1 function.

Journal Pre-proof

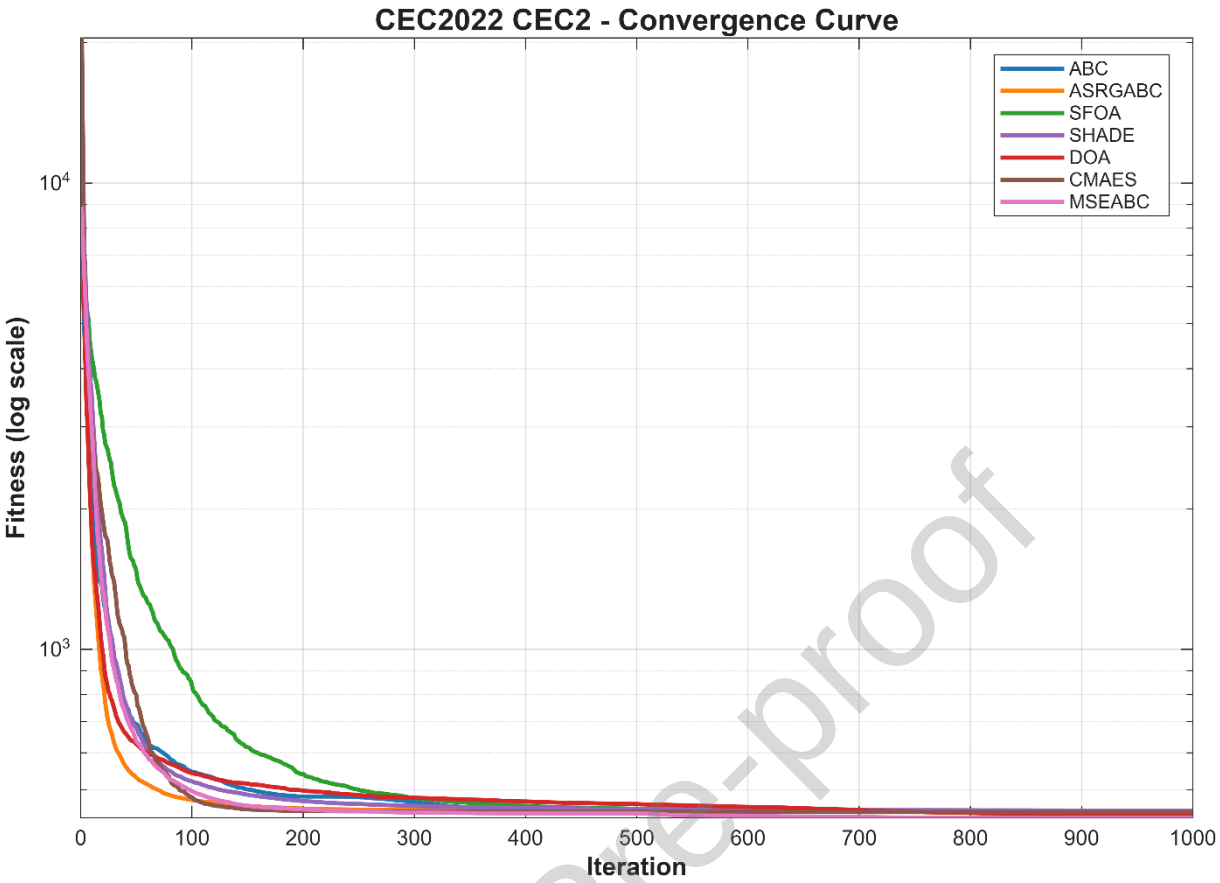


Fig. 5. Convergence curves of the algorithms on the CEC 2022 F2 function.

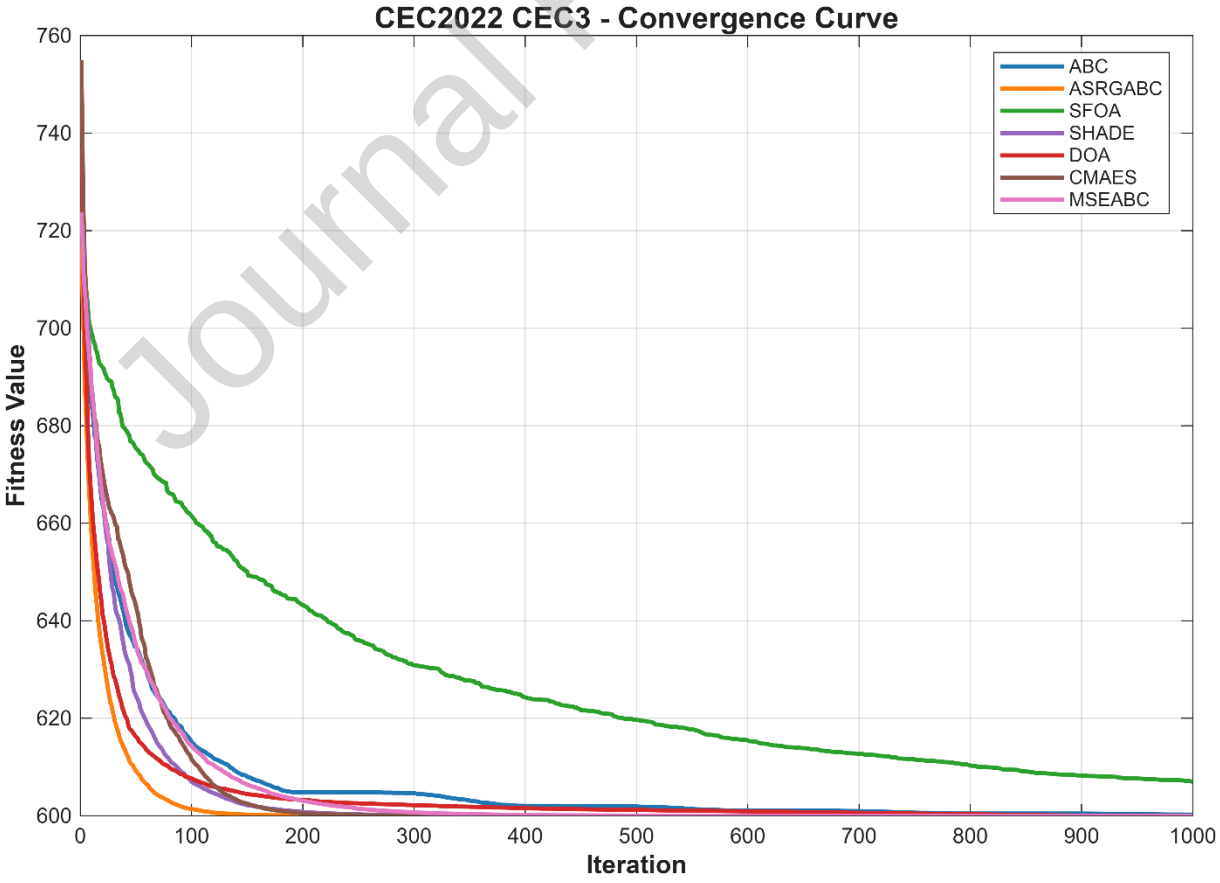


Fig. 6. Convergence curves of the algorithms on the CEC 2022 F3 function.

Journal Pre-proof

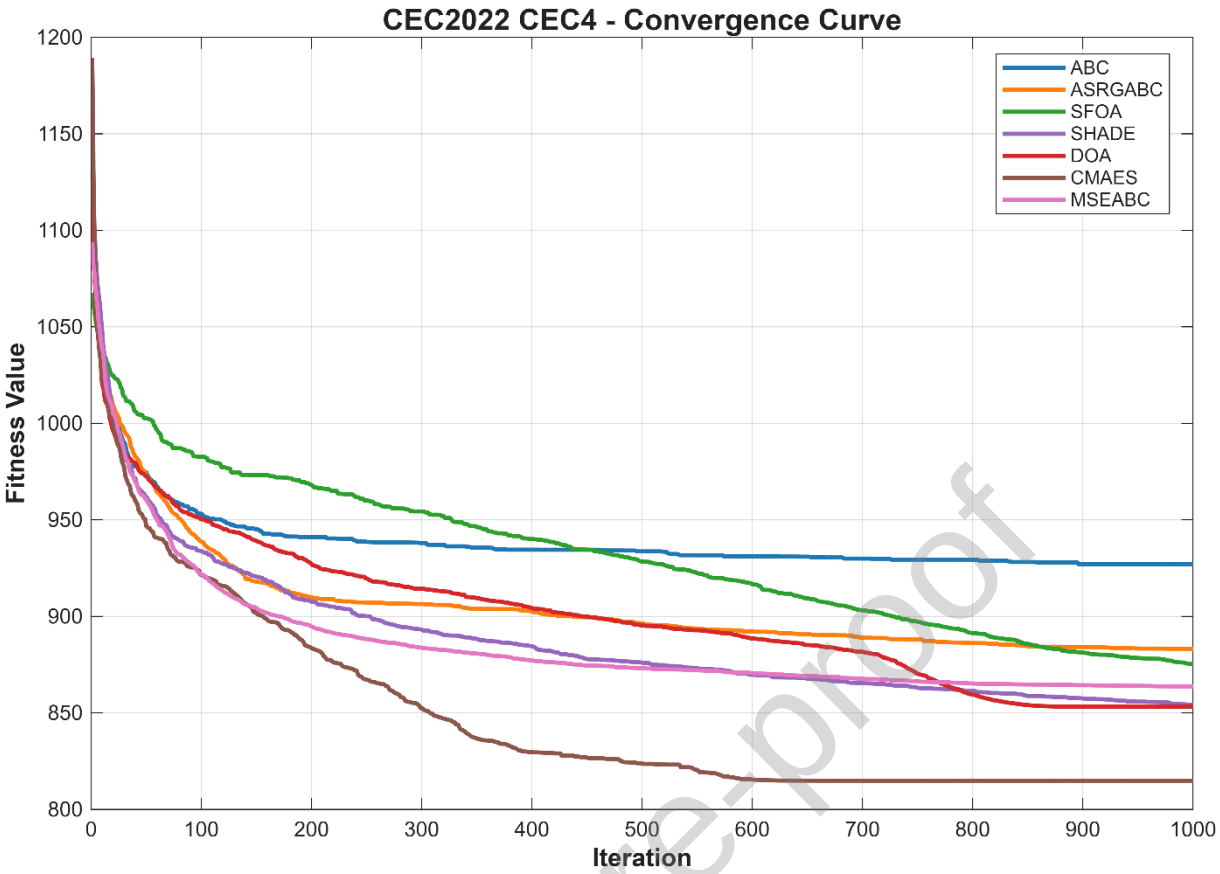


Fig. 7. Convergence curves of the algorithms on the CEC 2022 F4 function.

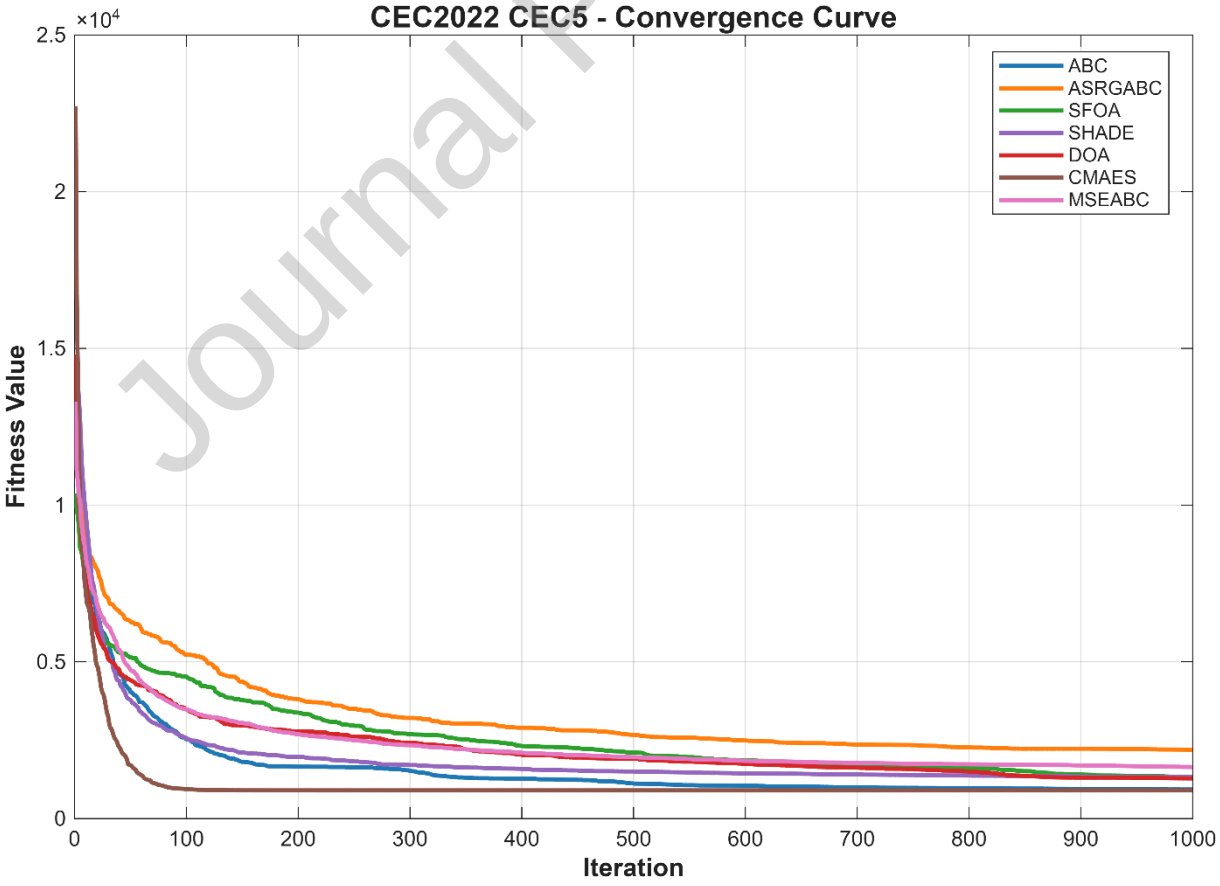


Fig. 8. Convergence curves of the algorithms on the CEC 2022 F5 function.

Journal Pre-proof

Journal Pre-proof

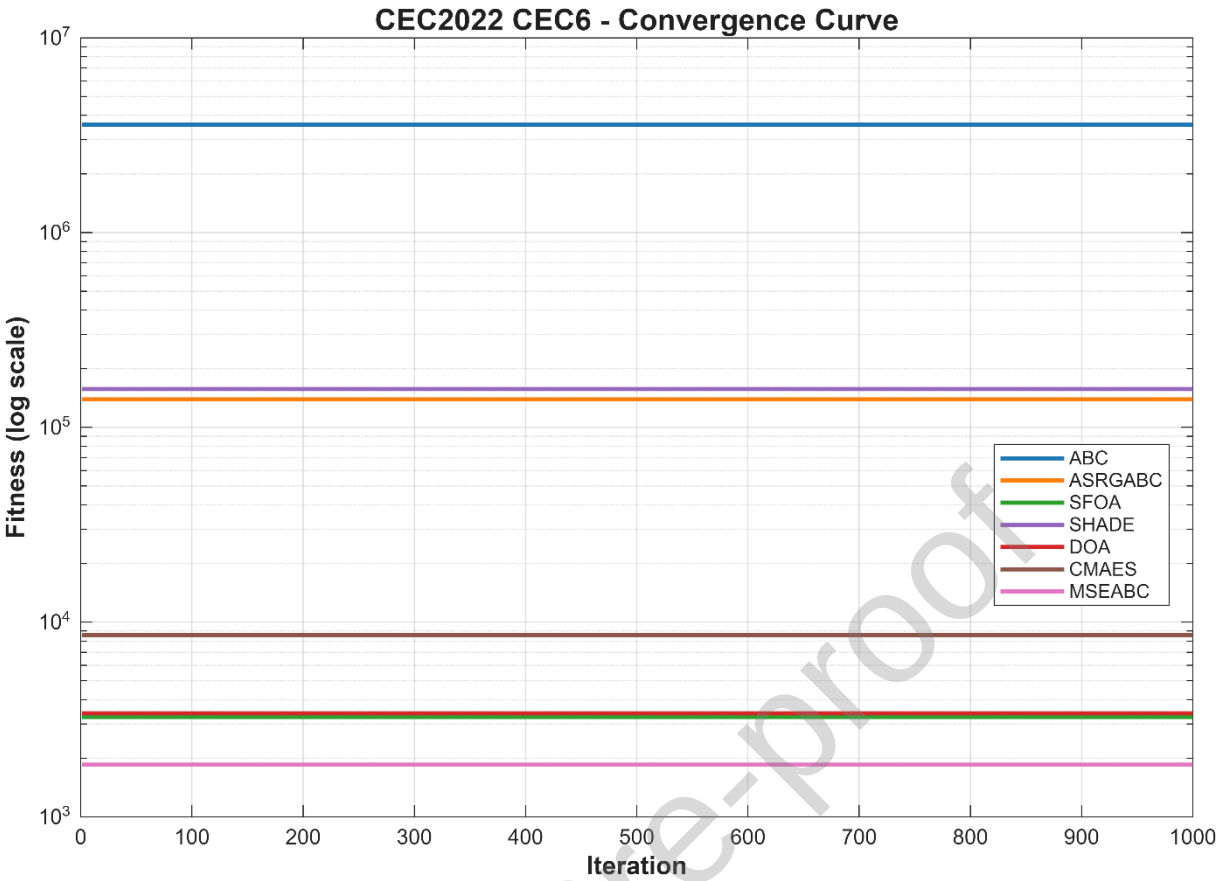


Fig. 9. Convergence curves of the algorithms on the CEC 2022 F6 function.

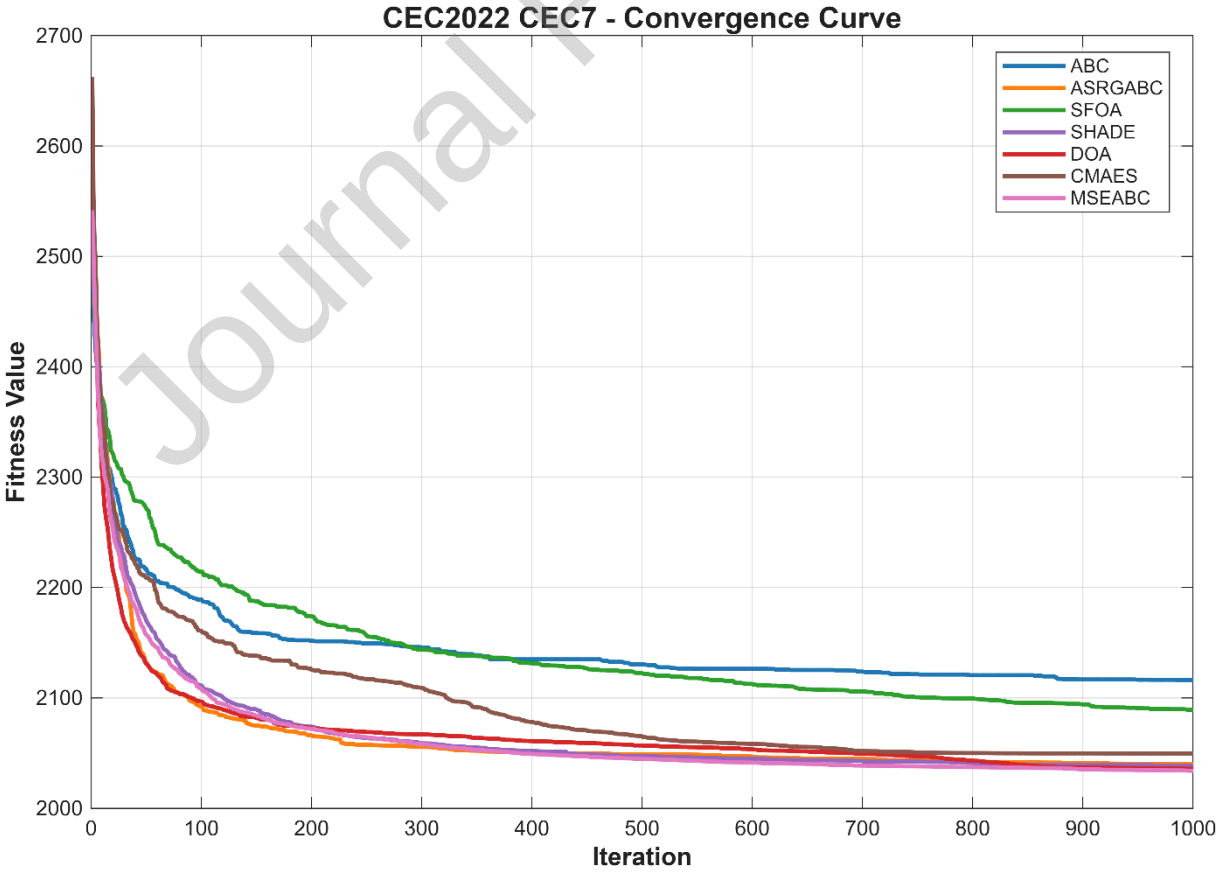


Fig. 10. Convergence curves of the algorithms on the CEC 2022 F7 function.

Journal Pre-proof

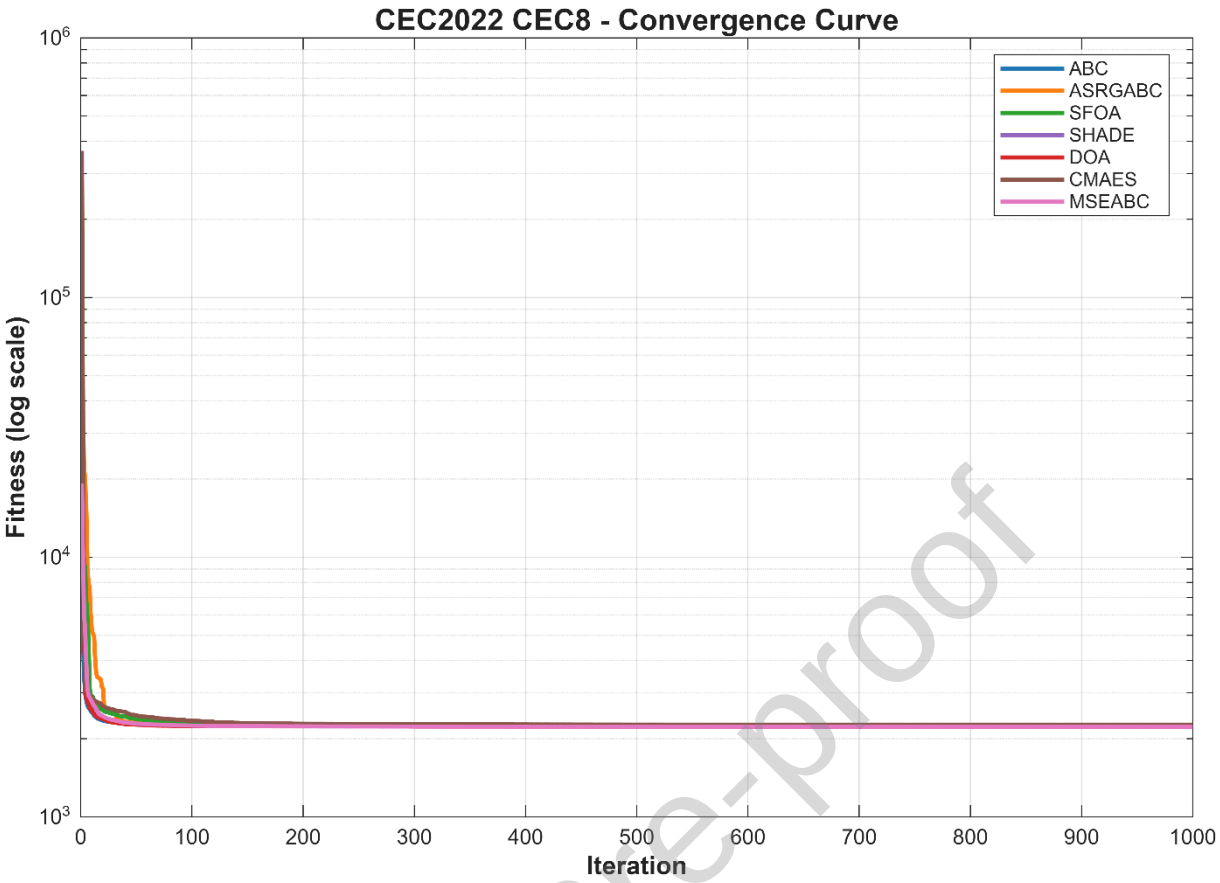


Fig. 11. Convergence curves of the algorithms on the CEC 2022 F8 function.

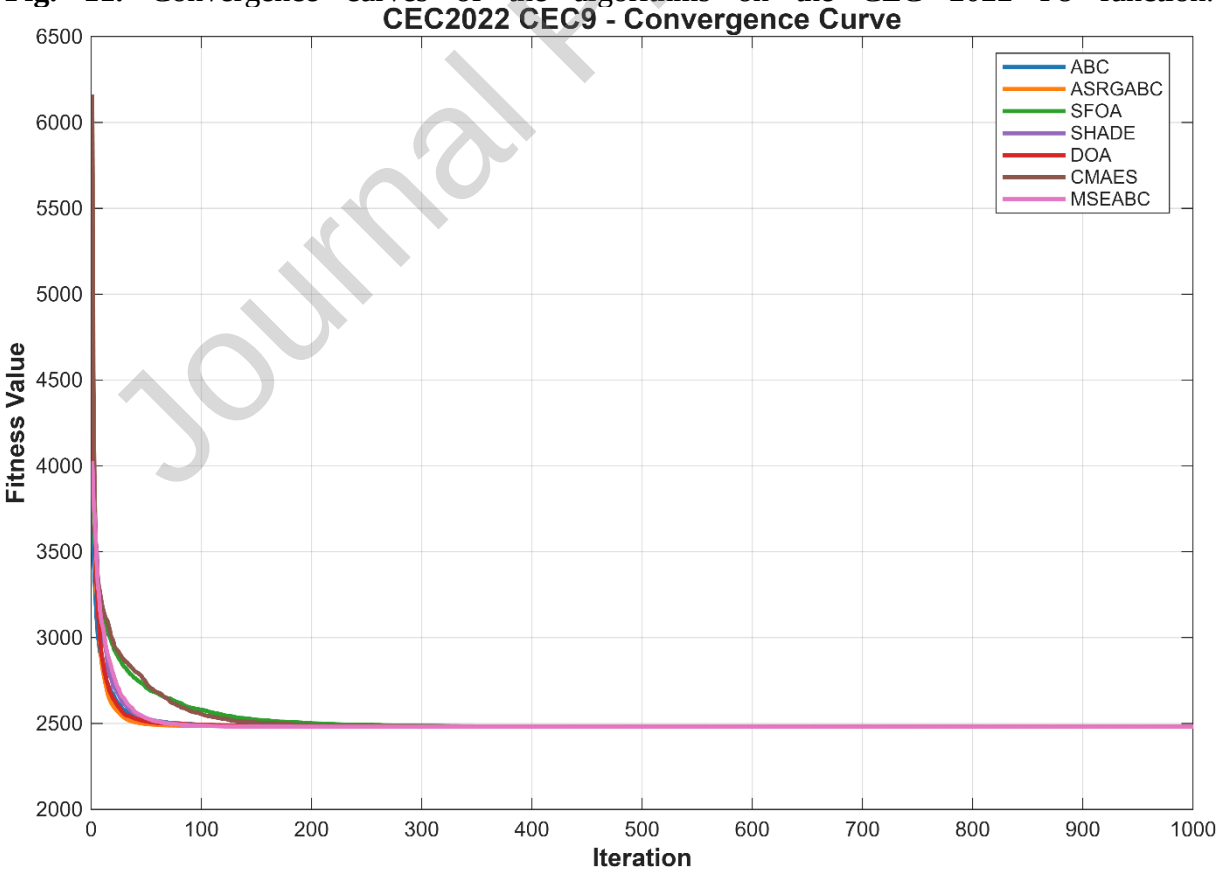


Fig. 12. Convergence curves of the algorithms on the CEC 2022 F9 function.

Journal Pre-proof

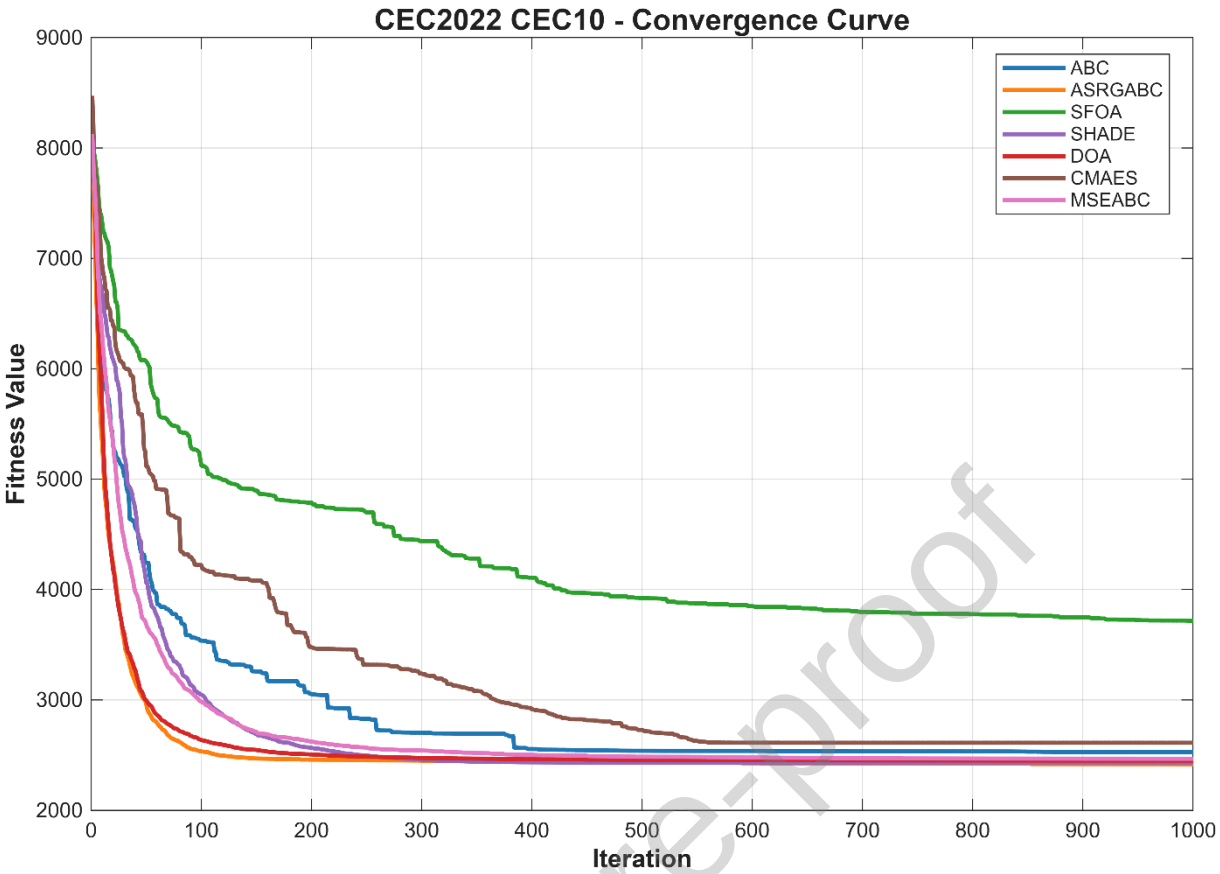


Fig. 13. Convergence curves of the algorithms on the CEC 2022 F10 function.

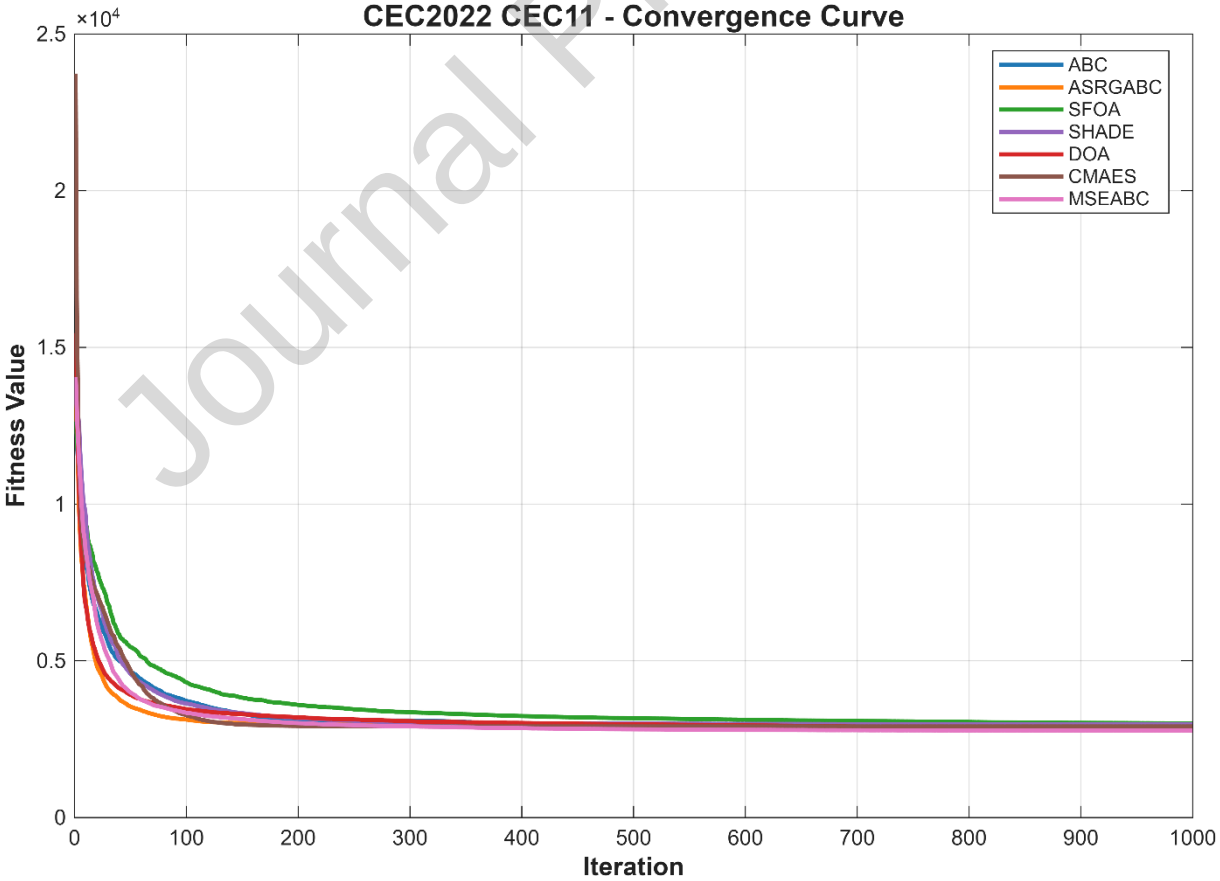


Fig. 14. Convergence curves of the algorithms on the CEC 2022 F11 function.

Journal Pre-proof

Journal Pre-proof

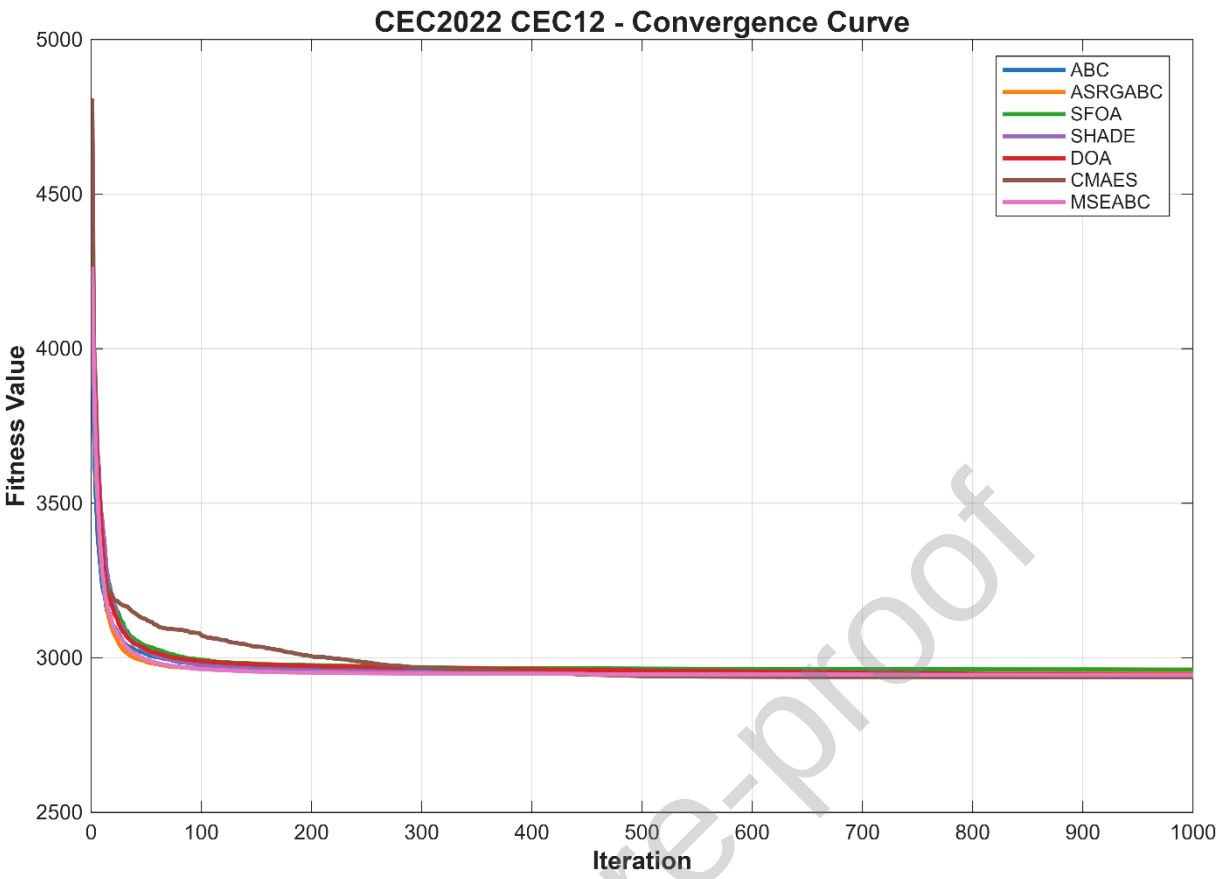


Fig. 15. Convergence curves of the algorithms on the CEC 2022 F12 function.

Journal Pre-proof

Declaration of Competing Interest

☒ The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☐ The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Highlights:

- MSE-ABC hybridizes ABC with starfish-inspired global search operators
- Momentum steering and dynamic step size optimize exploration-exploitation balance
- The algorithm solves CEC 2022, engineering design, and deep learning problems
- Its superiority over eight modern algorithms has been verified by statistical tests